



上海交通大学学位论文

基于大模型多 Agent 对抗的测试用例生成

姓 名： 常朋宇

学 号： 521021910970

导 师： 顾小东

学 院： 电子信息与电气工程学院

专业名称： 软件工程

申请学位层次： 学士

2025 年 5 月

A Dissertation Submitted to

Shanghai Jiao Tong University for the Degree of Bachelor

**LARGE LANGUAGE MODEL BASED
MULTI-AGENT ADVERSARIAL TEST CASE
GENERATION**

Author: Pengyu Chang

Supervisor: Xiaodong Gu

School of Computer Science

Shanghai Jiao Tong University

Shanghai, P.R.China

May, 2025

上海交通大学

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，已在文中以适当方式予以致谢。若在论文撰写过程中使用了人工智能工具，本人已遵循《上海交通大学关于在教育教学中使用 AI 的规范》，确保人工智能生成内容的应用场景、引用范围及标注方式均符合规定，并杜绝学术不端行为。本人完全知晓本声明的法律后果由本人承担。

学位论文作者签名：常明宇

日期：2025 年 5 月 20 日

上海交通大学

学位论文使用授权书

本人同意学校保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。

本学位论文属于：

☐ 公开论文

☒ 内部论文，保密 ☐ 1 年 / ☒ 2 年 / ☐ 3 年，过保密期后适用本授权书。

☐ 秘密论文，保密____年（不超过 10 年），过保密期后适用本授权书。

☐ 机密论文，保密____年（不超过 20 年），过保密期后适用本授权书。

（请在以上方框内选择打“√”）

学位论文作者签名：常明宇

日期：2025 年 5 月 20 日

指导教师签名：顾小东

日期：2025 年 5 月 20 日

摘要

软件测试是软件开发周期中一个重要而代价昂贵的步骤，其目标是确保软件的质量和可靠性，确保其按照预期的功能、性能 and 安全性要求正常运行。单元测试是软件测试的最底层，单元测试负责保证每个独立的单元能够按预期的方式运行并不产生错误。然而，编写高质量的单元测试对于软件工程师来说是枯燥乏味的工作。因此自动化测试用例生成工具不断地发展，基于搜索的软件测试工具虽然能生成有高覆盖率的测试用例，然而不能生成可读性强的测试。基于大语言模型的测试用例生成工具能模仿人类的编码风格，然而一开始却难以生成覆盖率高、通过编译的测试。因此研究者们往往通过迭代与反馈的方式来增加覆盖率，但反馈往往局限于覆盖率或编译报告等，较为静态，而且测试代码经常出现覆盖了代码行却并未对该代码行进行充分测试的情况。

因此，本文提出了一种基于大模型的多 Agent 对抗的测试用例生成方法。该方法由测试用例生成 Agent T 与变异体生成 Agent M 构成，通过覆盖率与变异得分驱动的交互式对抗循环，使得 T 不断优化测试用例以“杀死”由 M 提供的高挑战性变异体，同时 M 根据测试套件的盲区持续生成新的变异体，从而在多轮迭代中同时提升测试覆盖率与缺陷检出能力。最后在实验中与目前最先进的基于大语言模型的方法以及基于搜索的方法相比较。在 Defects4J 数据集上的实验表明，该方法的缺陷检出率相比目前最好的大语言模型方法提升了约 4.29%，相比基于搜索的方法提升了约 46.5%。

关键词：软件测试，大语言模型，测试用例生成，变异测试

ABSTRACT

Software testing is a critical yet costly phase of the software development lifecycle, intended to ensure that software meets its functional, performance, and security requirements with high quality and reliability. Unit testing—the lowest level of software testing—verifies that each individual component behaves as expected and does not introduce errors. However, authoring high-quality unit tests is often tedious and monotonous for software engineers. As a result, automated test-case generation tools have evolved continuously: while search-based testing tools can produce high-coverage test suites, they frequently lack readability; conversely, large language model (LLM)-based test generators can emulate human coding style but initially struggle to achieve both high coverage and compilable tests. Researchers therefore often adopt iterative, feedback-driven approaches to boost coverage, yet such feedback typically remains limited to static metrics like coverage reports or compilation logs, and tests may cover lines without adequately exercising them.

To address these limitations, this thesis proposes a multi-agent adversarial test-case generation framework driven by large models. The approach comprises a Test-Case Generation Agent (T) and a Mutant-Generation Agent (M), whose interactive adversarial loop is guided by both coverage and mutation scores. Agent T continuously refines its test cases to “kill” the challenging mutants produced by M, while Agent M persistently generates new mutants targeting the current blind spots of T’s test suite. Through multiple rounds of adversarial iteration, this method simultaneously enhances test coverage and defect-detection capability. Experimental results on the Defects4J dataset show that our approach improves defect-detection rates by 4.29% over the best existing LLM-based methods and by 46.5% over search-based methods.

Key words: software testing, large language model, test case generation, mutation testing

目 录

摘 要I

ABSTRACTII

第一章 绪论 1

 1.1 引言 1

 1.2 本文研究主要内容 2

 1.2.1 主要工作2

 1.2.2 论文章节安排3

 1.3 本章小结 4

第二章 相关技术调研5

 2.1 自动化测试用例生成 5

 2.1.1 传统方法5

 2.1.2 基于深度学习的方法 6

 2.1.3 基于大语言模型的方法 7

 2.2 变异测试 9

 2.2.1 传统变异测试9

 2.2.2 深度学习方法生成变异体 13

 2.2.3 基于大语言模型的方法生成变异体 13

 2.3 本章小结 14

第三章 方法设计15

 3.1 方法设计思路以及可行性分析 15

 3.1.1 方法设计思路15

 3.1.2 可行性分析16

 3.2 方法框架概述 18

3.3 测试用例生成 AGENT：测试用例初始生成	19
3.3.1 提示词生成与上下文收集	20
3.3.2 合成测试文件	21
3.3.3 修复未能编译的测试：基于规则的修复以及基于大语言模型的修复	22
3.4 变异体生成 AGENT：变异体初始生成	23
3.4.1 提示词生成与上下文	23
3.4.2 变异体存储	24
3.5 变异测试、数据收集以及流程控制	24
3.5.1 变异测试阶段与数据收集	24
3.5.2 流程控制及终止条件	25
3.6 测试用例生成 AGENT：测试用例后续生成	26
3.7 变异体生成 AGENT：变异体后续生成	26
3.7.1 针对覆盖率生成新变异体	26
3.7.2 根据存活变异体生成新变异体	26
3.8 本章小结	27
第四章 实验与评估	28
4.1 实验设计	28
4.1.1 数据集	28
4.1.2 对比基准	30
4.1.3 评价指标	30
4.1.4 环境配置	31
4.2 结果与分析	32
4.2.1 缺陷检出率	32
4.2.2 覆盖率	32
4.2.3 TOKEN 消耗	34
4.2.4 案例研究	35
4.3 本章小结	37
第五章 全文总结	38

5.1 主要结论 38

5.2 本文的不足与研究展望 38

 5.2.1 等价变异体识别 38

 5.2.2 存活变异体辅助测试用例生成的其他方法 39

 5.2.3 未来自动生成测试用例方法的展望 39

参 考 文 献.....41

完整提示词模板（附录 1） 45

致 谢.....47

第一章 绪论

1.1 引言

软件测试是软件开发周期中一个重要而代价昂贵的步骤，其目标是确保软件的质量和可靠性，确保其按照预期的功能、性能 and 安全性要求正常运行。单元测试构成了测试自动化金字塔中的最底层^[1-2]。健壮的代码库通常包含广泛的单元测试和集成测试。单元测试的目的是确保每个单独的单元（通常是代码中的函数或方法）在独立执行时能够按照预期的方式运行，并且没有产生错误。然而，对于开发人员来说，手动编写和维护高质量的单元测试是一个越来越冗长无聊的过程，以至于开发人员经常忽视测试^[3-6]。因此，自动化生成测试工具的开发便成为理所当然。

经典的测试方法例如 EvoSuite 是一个使用混合方法自动生成测试用例和断言的工具，旨在通过满足覆盖标准来系统地测试 Java 类，并帮助开发者验证软件行为正确性的测试工具^[7]。其采用基于搜索的测试用例生成方法，同样采用基于搜索的测试用例生成方法还有 Pynguin^[8]等。他们达成了当时行业的先进水平并且能够达到可观的覆盖率(coverage)。但是它们却有着生成测试的可读性和可理解性不足的问题^[9]。由此产生的测试需要大量的开发人员努力进行调试和理解，阻碍了整体采用。

大语言模型 (Large Language Model, LLM) 能够模仿人类的编码风格，通过合理的变量名称，函数名称，合理的数据选择，以及适时以及有帮助的注释提升生成的测试用例的可理解性，进而帮助人类工程师找到漏洞^[10]。大语言模型的最新进展塑造了 AI Agent 的新范式，即基于大模型的 Agent，基于大模型的多 Agent 协作的方法已经在软件工程中应用并且展示出显著的有效性^[11]。

与此同时，基于大模型多 Agent 的对抗方法的潜力也被发掘。多 Agents 对抗的方法在人工智能领域早已被使用，例如生成对抗网络(Generative Adversarial Network, GAN)采用了生成模型(Generator)和判别模型(Discriminator)对抗的思路，让生成模型能够产生越来越逼真的数据^[12]。而这一思想也与大模型结合应用，例如，在表格数据生成领域，MALLM-GAN 采用多 Agent 大语言模型作为生成对抗网络的框架，通过模仿对抗优化来生成高质量的表格数据^[13]。

变异测试(Mutation Testing, MT)和变异分析是软件测试领域常用的一种方法，有以下用法：测试集评估，测试集选择，测试集最小化，测试集生成，代码错误定位

等^[14]。变异测试是一种白盒测试技术，用于评估一个测试揭露代码错误的能力^[15]。变异测试涉及到注入人造的变化进入被测程序中以生成变异体，一个测试用例能够识别出变异体与原程序的差别称之为其“杀死”了这个变异体，一个测试用例能杀死的变异体越多，其识别真正的错误就越有效^[15]。

正如上文所提到传统的基于搜索的测试生成方法能够生成具有可观覆盖率的测试，然而基于大语言模型的方法虽然能够生成代码风格与人类工程师相近的测试代码，但在生成高覆盖率的测试上表现稍差，这是因为大模型往往关注总体语义，所覆盖的分支通常是常规的分支，需要通过迭代反馈，或是 HITS 采用切片的方式，让大模型在更小的范围内生成测试，这有效地增强了大模型的覆盖率^[16]。另外，目前针对大模型生成测试用例的研究对于生成效果的评价标准往往只有行/分支覆盖率(line/branch coverage)。而 MuTAP 能够从检出缺陷数量(#bug detected)和变异得分(Mutation Score, MS)来评价生成的测试用例，但是其选用的数据集 HumanEval 和 Refactory 较为简单，HumanEval 中包含 164 个人类编写的从简单到中等难度的问题^[17]，Refactory 则是 1710 个在 python 编程课下针对 5 个作业的课程的错误提交。虽然 Refactory 包含真实含有错误的代码，但并非项目级别的代码^[18]。因此，本文将注重于用大语言模型生成能力强的测试用例，让其不仅仅具有高的行/分支覆盖率，还应拥有在真实项目中的缺陷检测能力。

本文运用大语言模型于软件工程领域，探索大语言模型于软件测试中的应用，复现并测试多种测试用例生成方法，并且采用多 Agent 对抗的思路，其中一个 Agent 将利用变异测试优化生成的测试用例集，另一 Agent 将负责生成变异体，专门变异测试用例集仍未覆盖的地方。两 Agent 对抗，使得生成的变异体能高质量地评估测试用例集，也让生成的测试用例集合能够高效地分辨程序错误。

1.2 本文研究主要内容

1.2.1 主要工作

测试用例是软件开发的重要组成部分。随着大模型技术的发展，利用大模型自动生成测试用例已经成为主要的技术手段。当前基于大模型的测试用例生成方法常常关注程序的总体语义，所覆盖的分支通常是常规的分支，对于较难的测试分支通常难以覆盖。而且其揭露代码错误的能力在以往的文献中探究较少，主要探究覆盖率，包括行覆盖率以及分支覆盖率，而覆盖率与检测代码错误的效率是弱相关的^[15]。覆盖率指

标只检测代码行是否被测试，并不检测其能否检测出该代码行中可能蕴含的错误^[19]。而变异测试通过对代码行的变异，并检测测试用例能否检测出这种变异来检测测试用例检出错误的能力。

本文旨在研究一种基于大模型多 Agent 对抗的测试用例生成方法。由单元测试用例生成 Agent 生成测试用例，变异测试生成 Agent 负责生成具有挑战性的程序变体，两个 Agent 通过对抗学习，一个生成更难测试用例，另一个生成更具挑战性的变异体，从而逐步提高生成用例检测代码缺陷的能力，本文将在第三章对该方法作进一步详细说明。

变异测试和变异分析是软件测试领域常用的一种方法，变异测试是一种白盒测试技术，用于评估一个测试揭露代码错误的能力。本文将利用大模型生成变异测试的变异体，并将探索大模型生成的变异体以及其变异测试结果是否能够用于增强大模型生成测试用例的有效性。

另外，本文将比较本文提出的方法与其他基线测试用例生成方法，通过行覆盖率、分支覆盖率、错误检测率等多种指标进行比较。其他测试用例生成方法包括基于搜索的测试用例生成方法，例如 EvoSuite^[7]，也包括基于大语言模型的测试用例生成方法，例如 ChatUniTest^[9]。将主要在收集了真实世界中的 Java 项目的多个缺陷的 Defects4J^[20]数据集进行测试，以探究生成的测试用例揭露真实代码错误的能力。

1.2.2 论文章节安排

本文共五章节，各章内容如下：

第一章，绪论。这一章主要说明本文的意义和目的，并明确了本文研究主要内容，作为引入。

第二章，相关技术调研。这一章主要对本文的研究背景做了介绍，并对相关技术进行了了调研。

第三章，方法设计。这一章详细介绍本文提出的基于大模型多 Agent 对抗的测试用例生成方法设计，并对各模块的详细实现进行分析和介绍。

第四章，实验与评估。这一章介绍本文提出的方法所对比的基准，以及评价标准和数据集，并展示实验结果，最后，对实验结果进行分析。

第五章，全文总结。这一章对整个研究工作进行总结，并且提出本文仍存在的问题，以及提出进一步开展研究的见解与建议。

1.3 本章小结

本章节围绕研究动机，阐述了软件测试以及单元测试的重要性，并对现有的自动化测试工具（例如 EvoSuite^[7]，Pynguin^[8]）等在可理解性方面的不足进行了分析；接着介绍了大语言模型对该领域的潜力与作用。本章节还介绍了本文主要的研究内容，并对论文章节安排作了说明。

第二章 相关技术调研

2.1 自动化测试用例生成

测试用例是为具体的目标而开发的一组测试输入、执行条件和预期的结果。测试用例生成器是一种软件工具，具有较长的历史，它能接收输入源代码、测试准则、规格说明或数据结构定义，用这些输入以产生测试输入数据，有时还输出预期结果^[21]。下面，本文将对有关测试用例生成技术的研究进行介绍。

2.1.1 传统方法

传统的测试生成技术采用黑盒和白盒测试技术寻找程序的破坏输入。

随机/模糊技术例如 Randoop，是一个为 Java 生成单元测试的工具，采用反馈导向的随机测试生成，创建测试输入时收集执行反馈，以避免生成冗余和非法输入。Randoop 通过随机选择一个方法调用并从先前构建的序列中选择参数来逐步创建方法序列^[22]。一旦创建完成，新的序列就会被执行，并与合同集进行对比。其执行会导致违反合同(Contract)的序列将会被输出给用户。表现出正常行为的序列，即没有异常(Exception)且没有违反合同的序列，作为回归测试(Regression Test)输出。AFL++基于 AFL(American Fuzzy Lop)，AFL 是一种基于变异和覆盖率的测试用例生成器，它会变异一组测试用例，以达到程序中以前未探索过的点。此时，触发新覆盖率的测试用例会被保存为测试用例队列的一部分。它们通过保留提升覆盖率的输入来最大化测试覆盖率。AFL++相比 AFL，新增了许多功能，包括更智能的调度器，绕过路障，以及变异结构化输入等^[23]。

属性测试(Property based Testing)工具例如 QuickCheck 通过定义属性(properties)并随机生成输入来验证程序特性，开创了属性测试领域并催生众多语言的同类实现^[24]。Korat, 用于 Java 自动化测试，给定一个方法的形式规格说明(formal specification)，Korat 使用方法的前置条件自动生成所有的测试用例，直到给定的规模。然后 Korat 在每个测试用例上执行该方法，并使用方法的后置条件来检查输出的正确性^[25]。Hypothesis 则是一个成熟的、广泛使用的基于属性的 Python 测试库^[26]。

也有利用符号执行技术工具的方法，例如 PeX, 利用动态符号执行(Dynamic Symbolic Execution,DSE)生成高覆盖率的.NET 单元测试用例，并能自动注入测试代

码^[27]。KLEE 是一个基于 LLVM 编译基础设施的动态符号执行引擎,用于对程序进行深入路径探索并自动生成高覆盖率的测试用例^[28]。以及 Eclipser,将灰盒并行符号执行方法融入传统的模糊测试中,从而高效触达深层分支并发现隐藏漏洞^[29]。随机或模糊技术和符号执行技术的缺点在于,它们依赖于程序崩溃去找寻失败的输入,限制了它们可以测试程序的范围^[6]。

基于搜索的测试生成方法(Search based software testing,SBST)通过假设被测代码正确解决了这个问题^[6]。例如 EvoSuite,一个采用混合算法的针对 Java 程序的进化算法测试套件生成器,通过整体测试套件优化(Whole test suite optimization)策略,将所有分支覆盖目标视作一个整体,以遗传算子同时优化测试套件,并结合动态符号执行与变异驱动的断言合成,自动生成高覆盖率且带断言的 JUnit 测试^[7]。Pynguin 是一个面向 Python 的可扩展测试生成框架,通过搜索驱动技术自动生成高覆盖率的回归单元测试用例。基于搜索的测试生成方法能够生成具有合理断言的高覆盖率的单元测试用例^[8]。然而,基于搜索的测试生成方法难以生成可理解性高、具有良好代码风格的测试用例,这将不利于测试用例的维护,进一步地,这也阻碍了其在软件工程实践中的广泛使用。

2.1.2 基于深度学习的方法

基于深度学习的方法包括预训练或微调语言模型。这些方法把生成测试用例当作一个神经机器翻译(Neural Machine Translation,NMT)问题,其中输入是焦点方法,而输出是测试用例。

AthenaTest 在 github 上挖掘焦点方法和对应的单元测试,并添加到 Method2Test 数据集中,然后在英语和代码上预训练一个 BART Transformer,最后于单元测试生成任务上对其进行微调^[30]。A3Test 的研究人员发现 AthenaTest 在断言正确率和缺乏命名一致性以及测试签名验证上的缺陷,通过断言知识和验证命名一致性以及测试签名的机制增强基于深度学习的测试生成方法,并解决上述缺陷。其性能相较 AthenaTest 有明显提升^[31]。ConTest 使用通用的 Transformer 模型,利用代码的树的表示来生成断言^[32]。TeCo 是一种面向单元测试“下一条语句自动补全”任务的深度学习模型。与仅依赖语法信息的传统代码补全方法不同,TeCo 从测试方法上下文和被测代码中提取六类语义信息,以增强模型对代码执行逻辑的理解^[33]。CAT-LM 是一个 2.7B 参数的 GPT 风格的语言模型,在预训练阶段就考虑到了代码与测试文件的映射,并用更长的上下文窗口训练该模型^[34]。它们的结果表明,与传统的测试生成技术相比(例如 EvoSuite),由语言模型生成的测试用例更自然且更受开发者青

睐。

2.1.3 基于大语言模型的方法

在大语言模型推出后,便有研究者对其生成单元测试的能力进行了实证研究,2023 年的文献[35]表明大语言模型(GPT3.5 Turbo, StarCoder, Codex 三种代码生成大模型)在 HumanEval 数据集上面,能力与当前先进的测试生成工具有比较接近的性能。但是对于 EvoSuite SF110 基准测试,没有大模型达到超过 2%的覆盖率,远不及当前先进的测试生成工具如 EvoSuite。而且其生成的测试用例具有测试坏味道(test smell),例如空测试,重复断言等。说明了大语言模型当时在单元测试生成方面具有潜力,但仍需进一步发展。

下面将介绍一些基于大语言模型的测试用例生成方法。

TestPilot, 一个使用 LLM 在 Javascript/Typescript 上自动生成测试用例的工具。它给定一个被测程序(Program Under Test, PUT)作为输入,API 探索器(API explorer)识别出要测试的功能;文档挖掘者(document miner)提取关于这些功能的元数据;而提示生成器(prompt generator)、测试验证器(test validator)和提示优化器(prompt refiner)三者协同工作,构建用于生成测试的提示词,从语言模型的响应中组装完整的测试,运行这些测试以确定是否通过,并构建进一步的提示以生成更多的测试^[36]。

ChatUnitest, 一个基于 LLM 的自动生成单元测试的框架。其使用自适应的焦点方法上下文生成技术来解决上下文长度不足的问题,并且采用生成-验证-修复的流程来减少编译错误和运行时错误^[9]。

CODAMOSA 将大语言模型与传统的 SBST 相结合,论文探索了大语言模型能否用于解决 SBST 覆盖率停滞问题, CODAMOSA, 启动 SBST 并监控其覆盖率进展。当 CODAMOSA 发现覆盖率停滞时,它会识别 PUT 中覆盖率低的可调用函数。然后,它查询 Codex 以生成这些可调函数的测试。为了进一步构建这些测试, CODAMOSA 将 Codex 生成的原始字符序列反序列化为 SBST 内部的测试用例表示。这使得它能够利用 SBST 的变异操作和适应度函数来变异并评估 Codex 生成测试的质量。CODAMOSA 基于 Pynguin 构建,结果表明 CODAMOSA 相较于 Pynguin 具有有统计学意义的性能提升^[37]。

UTGen 则针对 SBST 生成的测试用例可理解性不足的问题,使用大模型对 EvoSuite 生成的测试用例进行改进,包括测试数据改进,更改变量名以及添加注释,最后为测试方法取一个名字。文献随后进行对照试验,探究这种改进能否使其更容易使用,更容易理解^[10]。

CODAMOSA 与 UTGen 分别以不同策略让 LLM 与 SBST 结合, 并分别解决了 SBST 不同的问题: CODAMOSA 解决了 SBST 搜索时覆盖率停滞, UTGen 解决了 SBST 生成的测试可理解性不足的问题。

SymPrompt 提出了一种“代码感知”的多阶段提示策略, 包括静态抽取路径约束, 构建调用上下文, 最后针对每条路径构建提示词并生成测试。研究表明, 这种提示策略能够大幅提升各个模型的表现^[38]。

MuTAP 首次提出了运用变异测试使用大语言模型生成测试用例的方法, 并且提出一种基于提示词的学习技术, 通过用存活的变异体增强 PUT 的初始测试用例提高测试用例的有效性。实验结果表明, 这种方法有效地增加了缺陷检出率和变异得分^[15]。

CoverUP 是一种面向 Python 回归测试的覆盖率驱动测试生成框架, 其核心思想是将代码覆盖率分析与大语言模型对话紧密结合, 通过迭代式反馈不断完善测试用例, 从而获得更高的行和分支覆盖率^[39]。

HITS 通过进一步地把焦点方法分解为切片(使用 LLM), 并让 LLM 为每个切片生成测试用例。该方法降低了分析范围, 使 LLM 在每个切片中更容易覆盖更多的行和分支^[16]。

TestForge 是一种用于测试生成的 Agent 方法, 能够节省成本地为复杂代码库中的大型文件生成单元测试。首先, 将测试生成视为一个迭代过程。TestForge 从零开始, 向底层的 LLM 模型提示待测代码。然后通过多次迭代逐步优化这些测试, 以针对未覆盖的行或变异分数较低的区域。其次, TestForge 利用详细的执行反馈——包括编译错误、运行时失败和未发现的代码段——作为其 Agent 循环的重要组成部分。它提供所有缺失覆盖的行作为 Agent 的输入, 而不是像先前方法那样选择子集。这使得 Agent 能够在一次迭代中规划多个测试用例, 提高效率。这种基于反馈的过程不仅提高了测试质量, 还提高了其在评估中的 Pass@1 (一次生成通过率)。最后, TestForge 在文件级别而非之前基于 LLM 或 Agent 的方法所考虑的单个方法级别运行, 这有效地节约了成本^[6]。

Meta 公司使用大语言模型自动改进现有的人工编写的测试, 并作了第一篇 LLM 生成代码的生产规模部署的报告, 证明大语言模型能够提高测试代码质量^[40]。

可以看出, 随着大模型能力的不断提升, 基于大语言模型的测试用例生成方法也在快速演进。从最初作为 SBST 的辅助, 到以多阶段 Prompt 为核心的策略, 再到最近的 Agent 驱动、覆盖率反馈迭代, 研究者不断探索如何更高效地让 LLM 聚焦

并理解代码的各个模块以及生成测试的目的。

2.2 变异测试

下面本文将介绍变异测试的基本原理和一些相关概念，以及利用深度学习/大模型进行变异测试的一些尝试。

2.2.1 传统变异测试

程序变异(program mutation)是从目标版本有目的地变更的计算机程序，也可称为变异体(mutant)。变异测试是一种测试技术，其中的两个或多个程序变异用相同的测试用例执行，以评价测试用例检测变异中差别的能力^[21]。进而变异测试可以用于评估测试用例揭露代码错误的能力，进而引向创造更高水平的测试用例。

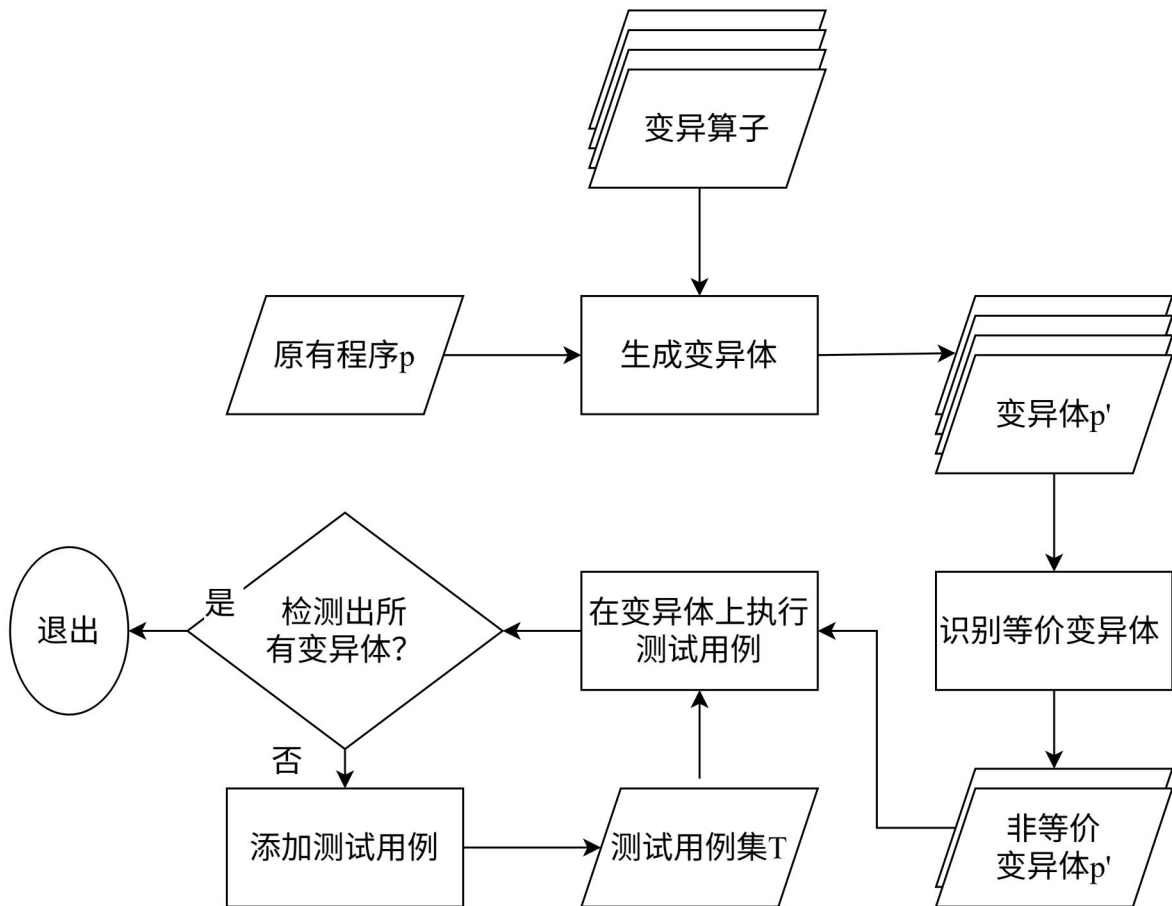


图 2-1 变异测试流程图

传统变异测试流程如图 2-1 所示，第一步给定原有程序 p 和测试用例集 T ，测试

人员根据被测程序的特性确定一系列变异算子(mutation operator), 变异算子是在符合语法的前提下对程序进行微小改动, 生成变异体的转换规则。例如, 一个常用的变异算子是将一个二元算数运算符替换为另一个(例: $c=a+b$ 可能被变异算子替换为 $c=a-b$, 而原有程序的其他部分不变)。一个典型的变异算子列表如表 2-1 所示(PIT 的基本变异算子^[19])。随后, 在原有程序 p 上执行变异算子以生成变异体。接着在生成的变异体中识别等价变异体。等价变异体是虽然在语法上与原有程序存在差距, 但是语义上相同的变异体。例如一个经典的 for 循环, 在循环内部不对循环变量 i 做任何更改, 且循环变量从 0 开始递增的情况下, 将终止条件判断由 $i<100$ 变异为 $i!=100$, 则这个变异体为等价变异体。

表 2-1 PIT 的基本变异算子^[19]

算子名	转换规则	例子
Cond. Bound.	用另一个关系运算符实例替换一个关系运算符实例 (单个替换)	$<\Rightarrow\leq$
Negate Cond.	反转一个关系运算符 (单个反转)	$==\Rightarrow!=$
Remove Cond.	用 true 或 false 替换条件分支	$\text{if}(\dots)\Rightarrow\text{if}(\text{True})$
Math	用另一个数值运算符替换一个数值运算符	$+\Rightarrow-$
Increments	将自增运算符替换为自减运算符 (或相反)	$++\Rightarrow--$
Invert Neg.	从变量中移除负号	$-a\Rightarrow a$
Inline Const.	将常量替换为另一个常量, 或对其进行增量操作	$1\Rightarrow 0, a\Rightarrow a+1$
Return Values	改变函数的返回值 (单次替换)	$\text{return } 0\Rightarrow\text{return } 1$
Void Meth. Call	删除对 void 方法的调用	$\text{void } m()\Rightarrow$

续表 2-1

算子名	转换规则	例子
Meth. Call	删除对非 void 方法的调用	int m()⇒
Constructor Call	将对构造函数的调用替换为 null	new c()⇒ null
Member Variable	将对变量的赋值替换为 Java 的默认值	a = 5 ⇒ a
Switch	将 switch 语句的分支标签替换为 Java 的默认分支 (default)	case x: ⇒ default:

检测等价变异体是不可判定问题^[41]，也是变异测试得到进一步推广和使用的主要障碍，本文第五章的“本文不足与展望”部分将提到有关内容以及相关研究的最新发展。在识别出等价变异体之后，在剩余的非等价变异体上执行测试用例集 T 中的测试用例，得到测试结果。并根据结果决定是终止变异测试，或是根据未被区分出的变异体设计新的测试用例，补充进入 T 中。

$$MS(M,T) = \frac{killed(M,T)}{|M| - eqv(M)} \tag{2-1}$$

当测试用例集 T 中的所有测试用例在生成的变异体上执行结束后，所有的变异体可以被划分死亡变异体(killed mutants)和存活变异体(survived mutants)。死亡变异体即存在测试用例 t 于测试用例集 T 中，在变异体 p'与原有程序上的执行结果不一致。存活变异体即不存在这样的测试用例 t。变异测试分析输出变异得分来评估测试用例集的缺陷检测能力。变异得分的计算公式如式（2-1）所示。其中 MS（M,T）表示变异体集合 M 在测试用例集合 T 上的变异评分。killed(M,T)表示变异体集合 M 中死亡变异体的数量。|M|表示变异体集合 M 中总变异体的数量。eqv(M)表示变异体集合 M 中等价变异体的数量。因此 MS 的取值介于 0 到 1 之间，MS 的取值越高，代表测试用例集检测实际缺陷的能力越强，最优目标即 MS 取值为 1，表明所有非等价变异体均被杀死^[42]。

传统变异测试依赖于两个由 DeMillo R 等人首先提出^[43]的假设。一是熟练程序员假设，假设熟练程序员编写出的有缺陷代码与正确代码非常接近。因此，变异测

试通过变异算子, 仅需对程序进行小幅度修改就可模拟熟练程序员可能出现的编程失误。二是耦合效应假设, 他们认为, 若测试用例能够检出简单缺陷, 则该测试用例也更有可能会检出更为复杂的缺陷。也有研究通过利用寻找真实的缺陷, 来证明变异分析对于检测代码缺陷的有效性, 并且其不但能检测出简单的错误, 更加复杂的错误也能检测出来^[44-45]。

变异测试是基于错误的测试技术, 主要用于测量测试集的质量, 还可以用于减少测试集的大小, 或用于指导测试数据的生成。

现有的变异测试方法大多是基于规则的:

JavaLanche 是一个面向 Java 程序的开源变异测试框架, 旨在提供高度自动化、高效能与高有效性的变异测试能力。在 Java 字节码层面生成与注入变异, 无需多次编译, 加快变异生成与测试流程。仅使用少量关键算子即可达到近乎全算子的效果, 减少无效变异体的数量。通过衡量变异对程序行为(动态不变量)的影响, Javalanche 能对等价变异体进行优先级排序与过滤, 减轻后续人工分析负担^[46]。

Major 是一个基于 javac 编译器插件的 Java 变异测试工具, 通过遍历和修改抽象语法树(Abstract Syntax Tree, AST)来注入变异, 所有变异都被编译进字节码, 运行时可动态启用或禁用, 无需重新编译。内置多组算子, 用于不同级别的变异, 包括运算符替换、值替换、语句删除等^[47]。

PIT 是一种实用的 Java 变异测试工具, 适用于实际代码库。PIT 运行速度快, 因为它基于字节码并优化了变异执行。此外, PIT 还非常稳健, 并且与开发工具高度集成, 可以通过命令行界面、Ant 或 Maven 调用。PIT 的基本变异算子如表 2-1 所示^[19]。

Proteum 在 C 程序上进行变异测试^[48], Milu^[49]是一个面向 C 语言的变异测试框架, 能够进行一阶和高阶变异(Higher Order Mutation Testing, HOM Testing)^[50]测试, 高阶变异体是通过多次应用变异算子产生的, 而一阶变异体仅应用一次变异算子。

AccMut 是基于 LLVM V3.8 构建的 C 程序的变异测试框架。Accmut 在 LLVM 中间表示(Intermediate Representation, IR)水平上生成变异体, 并整合了一些加速技术。支持 11 种变异算子^[51]。

WinMut 是大规模 C 程序的有效变异分析工具^[52]。

MutPy 是 Python 3.3 以上版本源程序的变异测试, 在 AST 级别应用变异。可以通过高阶变异和代码覆盖分析来增强变异测试过程^[53]。

2.2.2 深度学习生成变异体

与 2.1.2 中提到的深度学习生成测试用例相似, DeepMutation 将生成变异体的任务看作序列到序列(sequence-to-sequence)的神经翻译问题(Neural Machine Translation, NMT), 使用循环神经网络(Recurrent Neural Networks, RNN) 编码—解码(encoder-decoder)架构, 从大约 78.7 万条从真实项目中挖掘的缺陷样本中学习“变异”模式^[54]。而 LEAM 是一种面向软件变异测试的深度学习技术, 它通过从真实缺陷中学习来自动生成高质量的变异体, 并保证生成代码的语法正确性和语义有效性。LEAM 将方法转为抽象语法树以同时捕捉代码的结构信息和语义信息^[55]。但是这些方法在生成语法正确的变异体或是不同的变异体上受到了挑战。

我们可以发现, 在深度学习生成变异体的方法中, 变异算子的概念不复存在, 因为神经网络能够从真实缺陷中学习变异模式, 而变异算子本质上是一组人工定义的、固定的代码转换规则, 它们虽然简单易实现, 却往往只能覆盖有限的缺陷模式, 难以反映真实世界缺陷的多样性与语义上下文。相比之下, 深度学习通过从海量真实缺陷中学习, 能够自动挖掘出更丰富、更贴近实际错误的变异模式, 不再需要显式地设计算子规则, 也能生成更具语义相关性和上下文一致性的变异体, 从而显著提升变异测试的有效性和覆盖深度。也就是说, 变异测试的本质是用变异体“模仿”现实中的代码错误, 而变异体与真实代码错误的相似度越高, 变异测试的有效性就越高。这与变异测试被提出时候的假设相符。

2.2.3 基于大语言模型的方法生成变异体

基于预训练大语言模型的方法能够不需要训练的情况下就可转入一个新项目, 其中 μ Bert 使用 CodeBERT 预测被遮罩的代码元素来生成变异体^[56]。

LLMorpheus 通过反复地提示大语言模型在指定的 Javascript 代码片段注入错误, 让其生成多样化的错误。研究表明 LLMorpheus 能够生成多样化的变异体, 其中一些类似于现实世界的缺陷, 而这些缺陷是 StrykerJS 的标准变异算子无法创建的^[57]。

文献[58]较为全面的探索了大语言模型在生成变异体方面的能力, 并与最先进的基准进行比较, 例如 PIT, Major 等, 并且探索了多种模型生成变异体的能力, 包括开源大模型和闭源大模型。大模型使用较为基础的少样本学习提示词(few-shot prompt), 文献采用多种评价指标, 包括真实漏洞检出率, 耦合率等评价变异体与真实漏洞相似性的指标, 以及编译通过率, 无用变异率(包含重复的变异以及与未变异代码相同的变异)等可用性指标。其在 defects4J 和 Condefects 两个数据集上进行实验, 实验结果表明, 虽然基于规则的传统方法(PIT, major)在可用性指标上远

超大语言模型，因为基于规则的方法排除了通不过编译的变异算子，使得编译通过率达到了极高水平，而其设计也导致不会出现无用变异，而编译通过率最高的大模型（GPT-4o）只有 75.6% 的编译通过率。然而，大语言模型引入了更多的 AST 节点，表明了大语言模型生成的变异体具有更高的多样性。然而大语言模型在与真实漏洞相似性的指标上超越了其他方法，在真实错误检测能力方面，六种语言模型中有五种显著优于传统方法。特别是在 ConDefects^[59]上，GPT-4o 比传统方法 Major 高出近 29%，突显了语言模型在检测真实错误方面的优势。GPT-4 达到了最高的耦合率 44.1%，比所有传统方法，以及最好的其他方法 μ Bert，高出 2.5%。

另一篇文献[60]观察了使用 μ Bert 生成的变异体与现实中的通用漏洞披露 (Common Vulnerabilities and Exposures, CVE) 的不同耦合程度，研究发现， μ BERT 可以生成破坏相同测试的变异体，在 45 个漏洞中的 32 个原因与原漏洞相同。此外， μ BERT 还可以生成变异体，尽管原因不同，但同样破坏其他（剩余的）7 个漏洞的测试。总体而言，基于大型语言模型的变异体成功“强耦合”或“测试耦合”了 45 个漏洞中的 39 个。这证明了大型语言模型能够产生与漏洞相同方式偏离程序行为的变异体。

综上所述，基于大语言模型的方法在生成变异体方面具有巨大的潜力，相较于基于规则的变异算子，使用大语言模型的方法更多样化，更灵活，生成的变异体与真实代码错误相似度更高。这更有利于在代码开发的过程中，利用变异测试发现测试用例集的不足，并帮助我们构建一个更好的测试用例集。

2.3 本章小结

本章深入地介绍了本文的研究背景，围绕自动化测试用例生成与变异测试两大主题，系统梳理了相关技术的国内外研究背景与发展脉络。最先进的方法从传统方法到深度学习方法到最近基于大语言模型的方法的演变体现了软件工程智能化的趋势。

第三章 方法设计

3.1 方法设计思路以及可行性分析

3.1.1 方法设计思路

生成对抗网络(Generative Adversarial Network, GAN)采用了生成器(Generator)和判别器(Discriminator)对抗的思路,生成器,以随机噪声为输入,输出一条假样本 $G(z)$, 判别器以真样本或假样本为输入,输出该样本为真实样本的概率,双方互相对抗学习,让生成模型能够产生越来越逼真的数据^[12]。这一对抗思路在大模型时代也有应用,例如 MALLM-GAN 采用多 Agent 大语言模型作为生成对抗网络的框架,通过模仿对抗优化来生成高质量的表格数据^[13],其框架如图 3-1 所示。该框架采用一个可学习的表格分类器作为判别器,能够区分合成的表格和原有数据中的表格。图中的 θ 表示数据生成过程,冻结的大语言模型作为生成器和优化器,生成器根据 θ 和提供的少样本样例合成数据,优化器根据判别器得到的分数优化 θ 。MALLM-GAN 的对抗训练受到 GAN 的启发,但与传统 GAN 在本质上有所不同,它在一个自然语言优化空间中运行,使用大语言模型进行基于提示的生成,而缺乏形式化的数学保证。与 GAN 中基于梯度的优化不同, MALLM-GAN 的优化依赖于通过对抗优化与判别器对提示进行经验性精炼。由于缺乏数值梯度,理论收敛分析具有挑战性。但是实验表明随着提示的细化,判别器的准确性逐渐下降。这是最终收敛性的一个良好的现实替代。

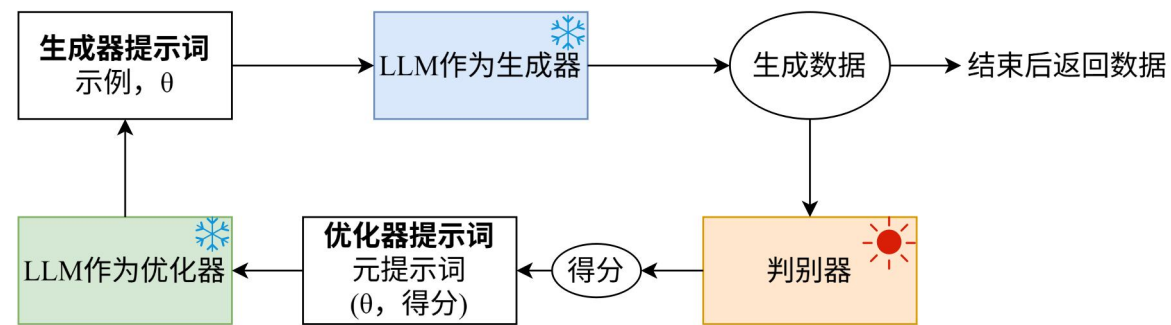


图 3-1 MALLM-GAN 方法示意图^[13]

MALLM-GAN 为本文提供了一个很好的多 Agent 对抗以及反馈的范例,在测

试用例生成领域，评价测试用例的主要有覆盖率指标和有效性指标（缺陷检出率、变异得分等），一个很好的指标是变异得分（公式 2-1，于 2.2 节），即测试用例能够区分并终止多少带有微小语义变动的变异程序，高变异得分意味着测试用例能有效捕捉代码中的潜在缺陷，从而在测试质量评估中具有更强的鉴别力。覆盖率指标虽然被广为使用，但是其只能说明测试用例“执行”了某段代码，并不能说明这段代码潜在的缺陷能被发现，而且覆盖率指标有一个固定的标准，而非像变异得分一样随着变异体生成器的不同或执行策略的不同而变化。

虽然 MuTAP 是第一个提出在基于大模型的测试用例生成中使用变异测试的方法。但是变异测试在 SBST 中已经有了应用。例如，在 Pynguin 的实现中，它利用 MutPy^[53]生成的变异体为测试生成断言，Pynguin 通过跟踪对象属性和函数返回值的变化，确定变异版本中哪些值与原始模块不同。对于这些值，Pynguin 生成断言，将原始模块返回的值解释为真实情况。因此，如果变异版本表现出与原始模块不同的行为，生成的断言能够终止这些变异版本^[8]。包括 EvoSuite 使用变异测试生成一组在最大化测试用例在类中揭示的种子缺陷数量的条件下精简的断言。精简的断言减少了重复断言，从而减少了测试坏味道^[7]。这些断言突出了当前行为的相关方面，帮助开发人员识别缺陷，并捕捉当前行为以防止回归错误。可以发现，变异测试往往在 SBST 中被用于生成断言。

尽管在 MuTAP 和 SBST 方法中被使用，但是现有方法大多是一次性静态流程，变异体生成器往往是规则驱动的（2.3 节中的传统方法），缺乏对测试用例和变异体的反馈优化，缺少利用测试覆盖盲区和存活变异体信息做更深层次探索的能力。而测试用例生成器也往往依赖于手工设计或简单的规则驱动，无法自动针对“难以杀死”的变异体持续改进。

因此本文提出一种基于大模型多 Agent 对抗的测试用例生成，具体来说测试用例生成器 Agent T 与变异体生成器 Agent M 进行对抗，并进行交互式反馈优化，形成对抗与反馈循环，最终使生成器生成更高质量的测试用例。

3.1.2 可行性分析

(1) 基础模型能力增强

现有大模型能力不断增强，以 deepseek-v3-0324 为例，如图 3-2 所示，其百科能力、数学、代码性能相较于 deepseek-v3 均有增强，其 LiveCodeBench 的 Pass@1 比率从 39.2% 上升至 49.2%，LiveCodeBench 不仅评估代码生成，还涵盖自我修复

（self-repair）与测试输出预测（test output prediction）等更广泛的场景^[61]，DeepSeek-V3-0324 在这些扩展能力上同样表现优异，因此代码阅读、代码理解、编写测试的能力也得到全面提升。有了更强大的大模型作为基础，本文的方法更有可能成功。

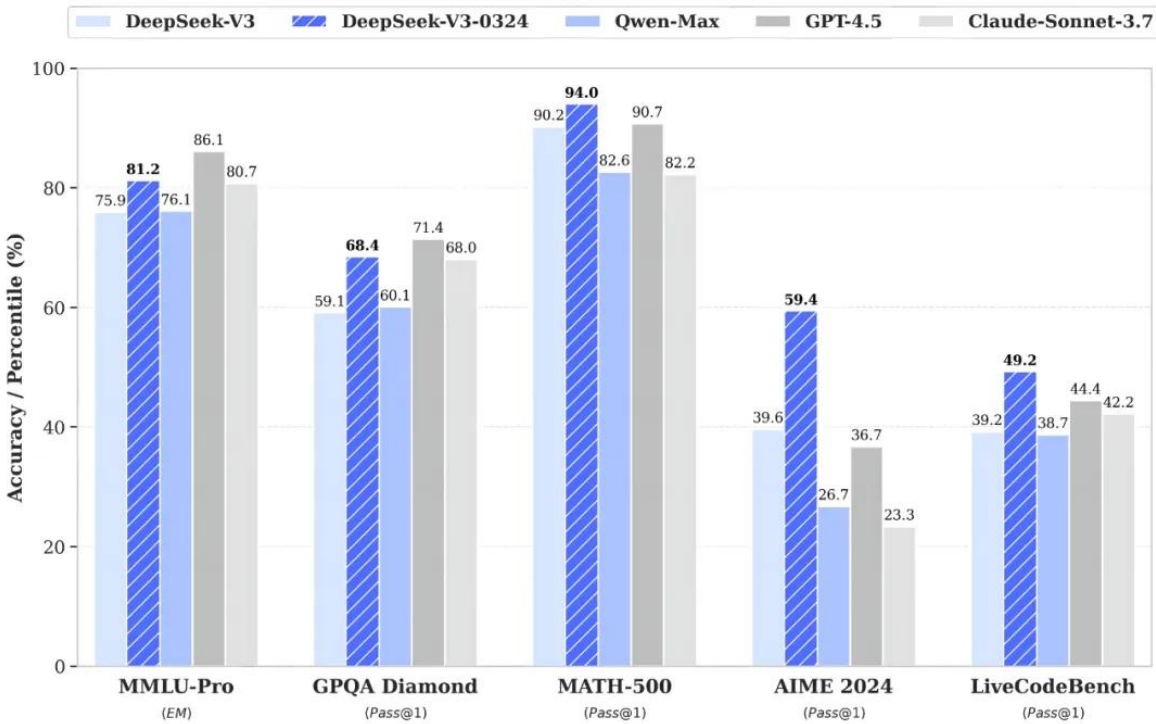


图 3-2 Deepseek-v3-0324 能力图^[62]

(2) 现有文献显示方法潜力

MuTAP 方法显示，使用存活变异体来增强提示词，因为这些变异体突出了测试用例在检测缺陷方面的局限性，该方法能够检测到多达 28%更多的由人编写的错误代码片段。并在变异得分指标上超出了所有其他方法^[15]。

文献[58]发现 LLM 生成的变异体在行为上更贴近真实缺陷，致使缺陷检出率提升约 25%，虽然 LLM 生成的变异体在可编译性、无用变异率和等价变异率方面劣于规则驱动方法，但其在“复现真实缺陷”与“多样性”方面的优势，为进一步优化变异测试提供了新思路。

上述文献以及其他工作^[7, 8, 15, 58, 60]充分证明了在 LLM 代码生成与测试用例/变异体生成中引入变异测试反馈的可行性与高潜力，为本文提出的多 Agent 对抗与交互式反馈优化方法奠定了坚实基础。

以上论述从基础模型能力和现有文献中验证了大模型多 Agent 对抗与变异测试反馈框架的可行性与潜力；方法和实现不断被优化，使得其最终结果得到提升。具体实验结果可在第四章查看。

3.2 方法框架概述

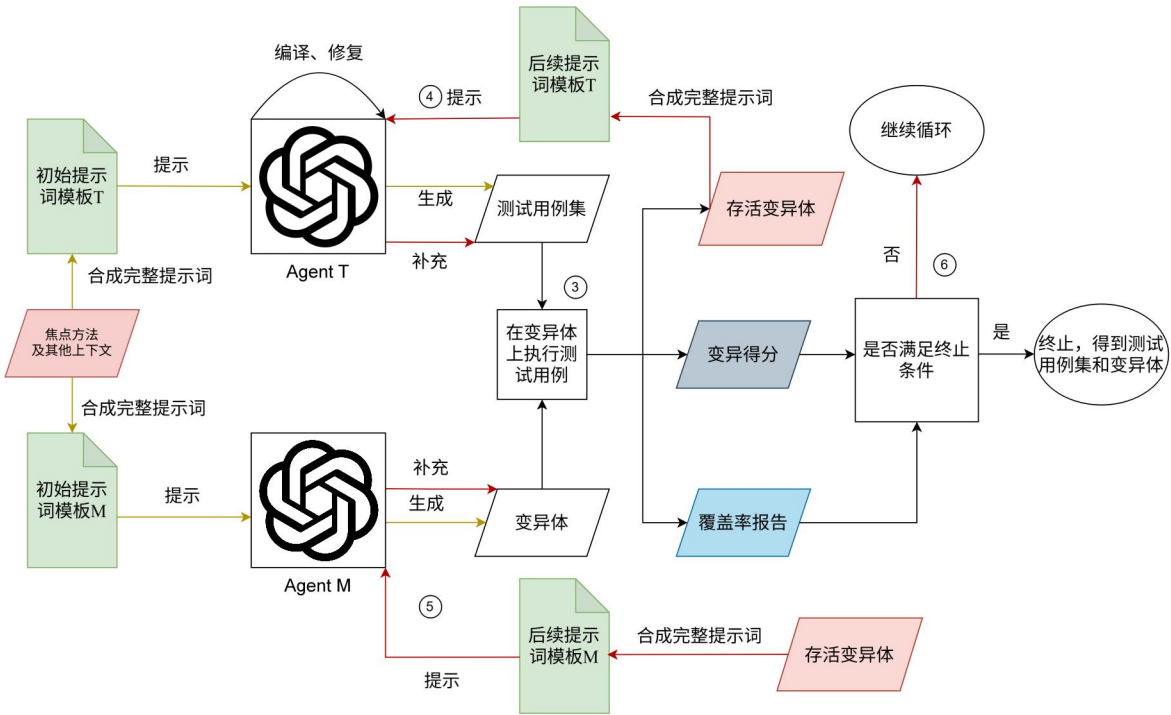


图 3-3 方法流程图

本方法是一个面向 Java 语言的基于大语言模型与变异测试的多 Agent 对抗生成测试用例的方法。本方法的核心框架在于两个 Agent——测试用例生成 Agent T 与变异体生成 Agent M 之间的对抗与反馈循环，这种循环体现在测试用例不断尝试杀死存活变异体，而变异体则持续生成更具挑战性的版本。并且这种互相对抗与反馈也将体现在覆盖率的不断上升与错误检测能力的不断上升以及变异得分的上下波动。方法流程如图 3-3 所示。

Agent T 与 Agent M 均基于大语言模型（非推理型），并可兼容包括 OpenAI GPT-4o 在内的主流系列模型；因其采用 Langchain 框架，几乎所有主流 LLM 都能无缝接入。GPT-4o、GPT3.5-Turbo 等模型也经过测试，最终综合考虑性能、响应时长等因素，在第四章的主要实验中，Agent T 与 Agent M 均选用 deepseek-v3。具体的

模型参数与调用细节，将在第四章中详细说明。

图中黄线所标注为仅第一次进入迭代前会进行的步骤，红线标注为仅后续迭代才会触及的步骤，黑线为无论是否为第一次进入迭代均有可能进行的步骤。

图中①所标注部分为测试用例生成 Agent 的测试用例初始生成部分，使用初始用例生成提示词来提示大语言模型，让其生成测试用例。该测试用例将经过编译修复后被保存为测试用例集 T。该部分将于 3.3 节进行详解。

图中②所标注部分为变异体生成 Agent 的变异体初始生成部分，在该部分中大语言模型会对指定的上下文进行变异体生成。生成的变异体被存入 json 文件中。该部分将于 3.4 节进行详解。

图中③所标注的部分为变异测试部分，将先前生成的变异体在测试用例集 T 上进行测试，对每个变异体 m 检测测试用例集 T 是否能区分该变异体与原程序。在进行变异测试后，会得到变异得分与存活变异体集合 M，另外，会将测试用例集在原程序上运行以得到覆盖率报告以及判定变异体是否存活的依据。之后变异得分和覆盖率将成为流程控制的关键参数。详细的讲解见 3.5 节。

图中④所标注的部分为测试用例生成 Agent 的后续用例生成，使用存活变异体强化提示词，让大模型通过提示词学习并推理存活变异体的行为，并尝试生成一些测试用例“杀死”它。详见 3.6 节。

图中⑤所标注的部分为变异体生成 Agent 的变异体后续生成，其主要包括两个部分，存活变异体驱动的变异体生成和覆盖率驱动的变异体生成。前者在存活变异体所变异的行上另作变异，后者在测试用例未覆盖的行作变异。详见 3.7 节。

图中⑥所标注的部分为终止条件判定，是否终止由总轮数，各自后续迭代进行轮数以及覆盖率和变异得分决定。详见 3.5.2 小节。

3.3 测试用例生成 Agent：测试用例初始生成

本节主要介绍测试用例生成 Agent 的初始测试用例生成部分，包括提示词生成与上下文收集与数据整理，以及生成-编译-修复的流程。

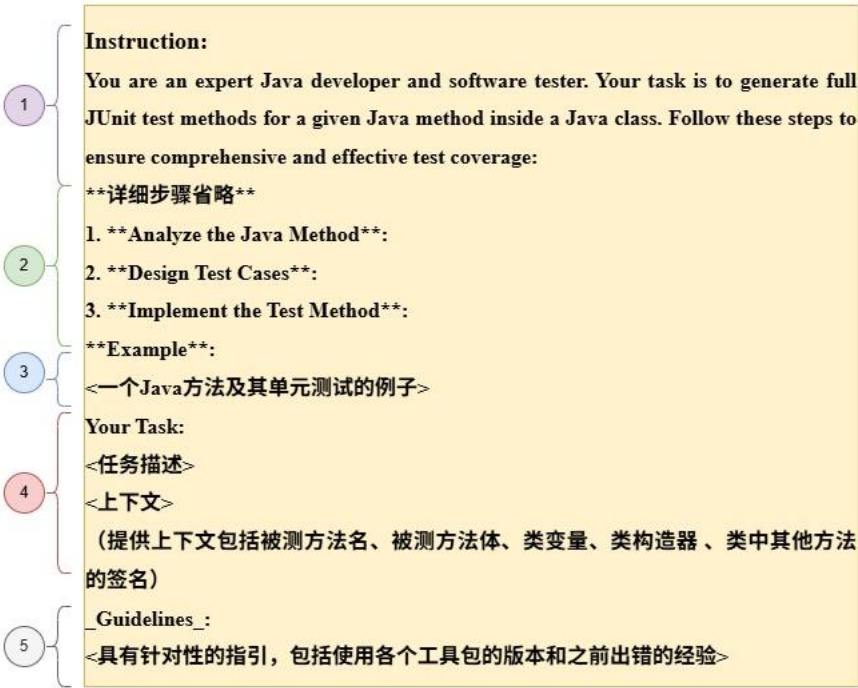


图 3-4 测试用例生成 Agent 提示词示意图

3.3.1 提示词生成与上下文收集

本文所描述的方法的提示词会用于提示 Deepseek-v3-0324 大模型，模型由 Deepseek API 平台提供。本文所描述的方法的提示词并不特别为 Deepseek-v3 设计，能够无缝接入其他大模型，只要该大模型被 Langchain 支持。每个不同步骤都设计了专门的提示词和指导，因为每个步骤拥有不同的任务和目标，不同的侧重点以及注意事项和经验。

本文所描述的方法参考了一些先进的提示词工程经验，并应用与本方法的提示词中^[63-65]。

思维链是一种通过在提示中展示模型的中间推理步骤，将复杂问题分解为若干易于处理的小任务，从而显著提升大型语言模型推理性能的方法^[63]。文献[66]，提出了零样本思维链的提示词，只要在提示词后加入“Let’s Think Step By Step”等类似的简单短语即可触发模型生成多步推理过程，从而提升回答准确率。模型显式地输出推理过程，或是让模型遵循一些步骤，都能够提升模型性能。

本文用于测试用例生成 Agent 的初次测试用例生成的提示词示意图见 3-4。在①中为 Agent 指定身份，Java 开发者与软件测试工程师，并为其初步描述任务。在②中采用类似 COT 的思想，为其描述应该遵循的步骤和指南。③中提供一些少样

本的例子，让大模型能够参考。④中详细描述任务以及提供任务所需的上下文。⑤中提供针对性的指引，包括应该使用的软件包的版本以及其他所需的信息。

本文的研究遵循了迭代的提示工程过程，其中每个提示的调整都是经过测试并根据结果的改进来决定是否调整提示词。⑤中的针对性指引也包括大模型之前经常犯错的部分，由人工提取并放入提示词中，显著减弱了模型犯错的几率。完整的提示词模板可见附录 2，相同 Agent 其他步骤的提示词不会再进行详细描述，仅描述不同点。

上下文方面，本文为 Agent T 提供以下信息：被测方法名、被测方法体、该方法所属类的成员变量、类的构造器，以及该类中其他方法的签名。这些上下文信息均由 Python 的 `javalang` 包从指定源码文件中提取，同时还收集了类的 `import` 和 `package` 声明，以便后续在提示中准确定位依赖。充分的上下文有助于大语言模型正确理解代码结构，避免因缺乏信息而误用 API 或错误调用函数。但为了让模型聚焦核心测试任务，也避免因一次性输入过多（例如整个类中所有方法的实现）而导致上下文过长、超出 Token 限制或推理效率下降，本文在提示中仅包含与待测方法直接相关的必要信息。

3.3.2 合成测试文件

大模型响应提示词并给出回复后，本文通过搜索 `@Test` 注解的方式，定位大模型给出的测试方法。本文将同一类中所有焦点方法生成的测试方法合成一个测试类，并命名为 `****Test.java`，****代表被测试焦点方法所在类的类名。这种做法是符合软件工程师的主流约定的。

合成过程中，首先形成测试类的 `import`，测试类的 `import` 首先应包含测试所需的软件包，如 `JUnit`，`Mockito` 等，其次，还应包含被测类的所有 `import`，否则可能导致在测试方法找不到对象等错误。其次形成测试类的 `package`，参考被测类的 `package` 进行修改，放入对应位置。

合成过程中，可能遇到测试方法命名相同的情况（这一点在后续步骤，使用存活变异体生成测试用例时更常见）。在相同的测试方法命名后续加阿拉伯数字序号，使其变得不重复。而不是像文献[10]方法描述的一样重新输入提示词直到大模型提供一个独一无二的测试方法名，因为本文的方法并不如其注重测试的可理解性，而重复输入提示词的方式不够稳定而且会提高时间与 token 消耗。

3.3.3 修复未能编译的测试：基于规则的修复以及基于大语言模型的修复

测试类文件合成后，将试着编译运行该测试。一项研究^[67]表明，多个程序员的编译失败率大约是正态分布，其中 Java 的编译失败率的中位数为 28.5%，而且失败率极低的开发者通常并不是该语言或项目的常规开发者，他们只做一些小改动，因此编译失败率低，而且该论文研究的并非初次编译，而是所有的编译行为。而大语言模型从零生成测试，其中还经过了拆分与重组，因此不应期待其生成的测试类第一次编译就能通过。

因此，需要一些方法修复编译错误。

ChatUniTest 为此提供了良好的实践，其有两种修复：基于规则的修复，以及基于 LLM 的修复，基于规则的修复用于修复一些常见的错误，例如：当 LLM 的回应超出 Token 限制时，可能会导致测试片段截断和语法错误。为了解决这一问题，ChatUniTest 首先添加必要的结束括号以验证结构。如果这一步失败，它会搜索“@Test”注解，截断其前的代码，并追加结束括号。如果修复失败或移除后没有剩余测试，该过程结束。另外假如“找不到符号”错误出现，ChatUniTest 将尝试通过简单地将焦点类中的所有导入语句复制到生成的测试中来解决该问题。基于 LLM 的修复则用于修复一些未曾想到的错误，ChatUniTest 会收集有关错误类型和消息的详细信息，将其与不正确的测试上下文整合，并使用预定义模板生成提示。如果经过指定轮次后测试仍未解决，整个过程将终止^[9]。

本文也拥有基于规则的修复以及基于 LLM 的修复，本文中已经提前为测试类添加了焦点类中的所有方法，本文中基于规则的方法主要针对以下几点：

1. 对因 LLM 响应截断或手动修改导致的大括号不匹配进行检测，统计打开与关闭大括号的差异，并在文件末尾补齐缺失的闭合括号，保证语法结构完整。
2. 自动添加分号，针对编译器报出的“expected ‘;’”、“not a statement”等错误
3. 针对测试片段中可能残留的无意义片段如“...”等自动清理。
4. 修复非法方法声明，若出现“invalid method declaration”、“非法的类型开始”等错误，通过大括号匹配和修改方法修饰符为 public void 尝试解决问题。

若基于规则的修复无法解决问题，将进入基于 LLM 的修复阶段，首先根据 @Test 注解以及大括号的位置，确定每个测试方法的起止行号，再把编译错误按行号映射到对应方法，否则归为“类级别错误”（如 import、字段声明等）。原始错误信息往往长而杂，本文把每个错误整理成 JSON 格式，保留行号与报错信息两个关键信息，去掉冗余的堆栈或文件路径，确保模型对出错位置和出错原因一目了然。

之后使用辅助函数给每行代码加上行号，并用 **Markdown** 的符号加粗标记突出显示出错行。这样不仅人看得清楚，LLM 也能更快地把注意力集中到真正的错误上下文。在上下文方面，除了图 3-4 中提到的上下文信息外，还提供上文中高亮处理后的需要修改的测试方法，以及与该需要修改的测试方法有关的报错信息。另外，在提示词最后指定的输出格式中（如图 3-5），分步输出的模板有助于模型输出其推理过程，激发其推理能力^[63, 66]。之前直接生成测试用例时未规定是因为直接生成测试用例任务较为简单直接。若经过 LLM 修复后，程序仍未成功修复，将递归地调用 LLM 修复，直到达到上限，经过多次测试，该上限定为 10 次，达到修复成功率与成本的平衡。

```
1. Tested method analysis:
The method being tested by our method is intended to .....

2. Error analysis:
The compilation error is..., which could be caused by .....

3. Test method analysis:
The test method line ... is causing the compilation error , we can fix the error by .....

4. Generate the test case:
...
<fixed test method>
...
```

图 3-5 指定输出格式示意图

若还没能通过编译，最终的解决方案是删除报错的行，直到通过编译。通过编译后，将会把该测试文件保存下来。

3.4 变异体生成 Agent：变异体初始生成

3.4.1 提示词生成与上下文

在文献[58]中探究了提示词与上下文对大语言模型生成变异体的质量产生的影响，结果表明，给出完整方法作为上下文，并且添加几个例子作为少样本学习样例效果最好，然而，添加测试用例进入提示词中，是表现最差的。推测因生成变异体是较为简单的任务，而测试用例与变异体的关系较远，而且推理起来较为复杂，因此其效果反而不如不添加。因此，在该提示词中也采用类似的策略，为 Agent 提

供完整方法作为上下文，以及少样本学习样例。另外，在提示词部分，显示地要求大模型不要生成与原来等价的变异体。提示词示意图见图 3-6。

```
<Original Code>

Above is the original code. your task is to generate x mutants in original code(notice:mutant refers to
mutant in software engineering, i.e. making subtle alterations to the original code) in :

<examples>

#Requirement:

<detailed_requirements>
```

图 3-6 变异体生成 Agent 提示词示意图

3.4.2 变异体存储

在提示词的#Requirement 部分，要求变异体生成 Agent 返回格式为 JSON 格式，以方便存储和后续处理，包括”precode”代表变异前代码，”aftercode”代表变异后代码，随后程序将在对应文件中查询 precode 的行号并加入，最终存储的变异体 JSON 字段包括 id,precode,aftercode,line,file。以此作为还未处理的变异体的存储，等待下一步变异测试进行使用。

3.5 变异测试、数据收集以及流程控制

3.5.1 变异测试阶段与数据收集

变异测试的原理以及相关术语详见 2.2 节，记变异体生成 Agent 生成的变异体集合为 M ，测试用例生成 Agent 生成的测试用例集为 T ，记未经变异的程序为 p ，对 $m \in M$ ，其在 T 上运行，当且仅当 m 能够成功运行（未出现编译错误、超时）且 m 失败通过 T 中用例的集合为 $F(m, T)$ ，与 p 失败用例的集合 $F(p, T)$ 完全相同，则 m 为存活变异体。此处存活变异体的定义与传统变异测试的定义并无不同，但由于大语言模型生成的可能的错误断言或其他可能导致运行时错误的问题，不能够保证此处未经编译的程序失败通过的测试方法数为最少，即无法保证 $F(p, T) = \emptyset$ 。

由于不可编译的变异体的存在，在本方法中，变异得分的计算略有不同，具体来说如式 3-1 所示。不可编译的变异体既不属于可杀变异体，也不属于存活变异体，直接被丢弃。

$$MS(M,T) = \frac{killed(M,T)}{killed(M,T) + survived(M,T)} \quad (3-1)$$

在变异测试后，便可计算变异得分，以及在测试未经编译的程序 p 的时候，可以得到行覆盖率。行覆盖率是指代码中有多少行被测试执行过。它计算的是已执行的代码行数占总代码行数的百分比。例如，如果一个程序有 100 行代码，其中 90 行被执行过，那么行覆盖率就是 90%。在变异测试后，变异体存储的 JSON 中会额外增加词条，`survived` 和 `failing tests`，表示该变异体是否于上次变异测试中生存，以及该变异体失败的测试。

3.5.2 流程控制及终止条件

得到上述信息后，根据变异得分 MS ，总轮数，以及行覆盖率 $Coverage$ 来决定下一步。超参数 MS_{target} ， $Coverage_{target}$ ； MS_{target} 表示期望的变异得分， $Coverage_{target}$ 表示期望的覆盖率。另外还有 R_t 代表测试用例后续生成轮数，与 R_m 代表变异体后续生成轮数。

经历一次变异测试则总轮数加一，若总轮数大于或等于 4 且 $R_m > R_t$ ，则停止程序，并将保存的测试用例作为最终结果。若总轮数大于或等于 4 且 $R_t < R_m$ ，则进行一次测试用例后续生成后停止程序，这是为了防止变异体生成的浪费，使得生成的大部分存活变异体都被测试用例生成 Agent 所知。

若总轮数小于 4 且 $MS < MS_{target}$ ，说明测试用例强度不足，下一步将执行 3.6 节的测试用例后续生成。若 $MS > MS_{target}$ 且 $Coverage < Coverage_{target}$ ，则执行 3.7 节的变异体后续生成。若 $MS > MS_{target}$ 且 $Coverage > Coverage_{target}$ ，则满足提前终止条件，此时可以提前终止。

为了在测试质量与执行成本之间取得平衡，本文将变异测试的总迭代轮数上限定为 4 轮，理由如下：

随着每一次迭代，测试用例生成 Agent 会基于现有存活变异体产生新的测试用例，变异体生成 Agent 也会产生新的变异版本。实际观测中，当迭代轮数超过 4 轮后，测试用例和变异体的总量会出现近似指数级增长，导致后续每轮的编译、执行的开销急剧攀升，显著拉长整个变异测试的总耗时。当总轮数达到或超过 4 且变异得分仍未达到预期阈值时，可能并非是测试用例的不足，也有可能是等价变异体

占比过大，因此继续执行会导致 Token 和时间的大量浪费。

3.6 测试用例生成 Agent：测试用例后续生成

测试用例生成 Agent 将会针对每个存活变异体生成尝试杀死该变异体的测试用例，所使用的提示词与已经在 3.3 节讲述过的相似，其中上下文还包括变异体的信息，将给出变异前的代码与变异后的代码，另外，也添加了 3.3.3 中的指定的思维链式的输出格式。在大模型给出响应后，测试方法将经历一个与 3.3.2 章节中描述相似的过程将新生成的测试方法与原有的其他方法合成一个测试文件，不过此时 `package` 和 `import` 已经在测试 Java 文件中保存下来，不需要通过原文件获取。另外，若遇到 LLM 修复不通过的情况，对于测试用例后续生成来说，直接跳过该存活变异体的测试用例生成，进行下一个。这是为了防止重复生成，然而发现该存活变异体为等价变异体的情况。当全部存活变异体都经过了一次生成后，将回到 3.5 节进行变异测试。

3.7 变异体生成 Agent：变异体后续生成

在完成初始变异体生成和变异测试后，系统既已获得测试用例对原程序的行覆盖率报告，也已收集到存活的变异体集合。变异体生成 Agent 进入后续生成阶段，采用两种互补策略：针对覆盖率生成新变异体，根据存活变异体生成新变异体。

3.7.1 针对覆盖率生成新变异体

针对覆盖率生成新变异体是一个简单直接的方法，因为覆盖率揭示了一个代码行是否在测试过程中被运行。在未被覆盖的代码行上生成一个新的变异体能够确保该变异体不会被旧有的测试用例杀死，也能够在今后的测试用例生成中用于提示测试用例生成 Agent，引导其增加覆盖率，覆盖之前未被覆盖的代码行。

3.7.2 根据存活变异体生成新变异体

然而，光针对覆盖率生成新变异体并不足以给测试用例生成 Agent 以足够的信息，因为覆盖到的地方，仍有可能潜藏漏洞，举一个显而易见的例子，假如一个测试用例没有断言，将几乎无法分辨漏洞，当覆盖率不断提高的时候，变异体后续生成的空间也不断减小。此时，存活变异体所在的行可能暗示测试用例并未充分对该

行进行测试。因此，可以根据存活变异体来生成新变异体。

为了减少大模型生成重复的变异体，本文按行将存活变异体分组放入提示词中，并显式地提示大模型：不要生成与已有变异体重复的变异体。

3.8 本章小结

本章围绕基于大语言模型的多 Agent 对抗测试用例生成方法展开论述：首先在 3.1 节提出设计动机与可行性分析，借鉴 GAN 的生成器—判别器对抗思想，将测试用例生成 Agent 与变异体生成 Agent 进行交互反馈；并结合 MuTAP、Pynguin、EvoSuite 中变异测试在 SBST 领域的成功应用，以及 deepseek-v3-0324 在代码生成与自修复任务上的性能提升，论证了方法在理论与实践层面的可行性^[7, 8, 12, 13, 15, 58]。3.2 节给出整体框架概述，展示 Agent T 与 Agent M 在初始化生成、变异测试、迭代反馈和终止判定等核心环节的流程与数据流。

3.3 节详细介绍测试用例生成 Agent 的初始用例生成：通过 Langchain 自动提取被测方法签名、实现逻辑、构造函数及依赖上下文，结合少样本示例与思维链提示词设计，引导大模型生成高质量测试方法；并引入基于规则与 LLM 的两阶段编译修复策略，解决常见语法或符号缺失问题，保证测试类顺利编译。3.4 节阐述变异体生成 Agent 的初始变异体生成：在提示词中提供完整方法上下文与示例变异体，实现 precode、aftercode、line 等字段的规范化存储，为后续变异测试做准备。

3.5 节聚焦变异测试与数据收集：解释基于大模型特性的存活变异体判定规则与变异得分计算公式，并结合行覆盖率报告和迭代轮数阈值，设计流程控制与终止条件，通过轮次数上限和得分 / 覆盖率双重条件，精细管理对抗流程。3.6 节描述测试用例后续生成策略：针对每个存活变异体构造提示词，尝试生成能“杀死”该变异体的测试用例，不断提升测试套件的缺陷检出能力；对无法修复的变异体直接跳过，避免重复浪费。3.7 节总结变异体后续生成策略：分别从覆盖率盲区和存活变异体两方面生成新变异体，既能揭示测试盲点，也能加剧对抗难度，保证变异体集合的持续演进。

综上所述，本章主要介绍了方法的理论基础以及全流程设计。第四章将会以实验探究该方法的能力，第五章会总结全文，并提出该方法仍存在的一些不足。

第四章 实验与评估

4.1 实验设计

本节首先介绍实验所用的数据集选取及其基本特征，然后说明对比基线和评价指标的设计，最后给出实验环境的配置。

4.1.1 数据集

为了在真实软件项目级别验证所提方法的有效性，本研究选取 Defects4J 数据集进行实验。Defects4J 由 Just R 等人创建，旨在为软件测试研究提供可复现的真实缺陷样本^[20]。其 V2.0.1 包含来自各个主流 Java 开源项目的 835 个真实缺陷（见表 4-1），每个缺陷均满足以下特点：

可复现性:每个缺陷均配有至少一个测试用例，该测试在修复版本(fixed version)上通过、在缺陷版本(bug version)上失败。该测试用例被称为触发测试(trigger test)。

隔离性:修复版本与缺陷版本差异仅包含修复本身，无其他改动。

完整性:每个项目均附带完整的测试套件，便于后续覆盖率与变异测试分析。Defects4J 同时提供了测试执行框架，研究者可方便地切换至缺陷版本与修复版本，执行测试覆盖率、变异得分等分析操作，并保证实验结果的可重复性和可比较性。在本研究中，基于该框架，对选择的七个项目未被弃用的缺陷依次运行本文提出的方法与其他基线方法，以评估所提方法与其他基线方法在不同项目下的性能^[20]。

表 4-1 Defects4J2.0.1 版本包含项目及缺陷信息^[20]

标识符	项目名称	缺陷数量	活跃缺陷编号	已废弃缺陷编号(*)
Chart	jfreechart	26	1-26	None
Cli	commons-cli	39	1-5,7-40	6
Closure	closure-compiler	174	1-62,64-92,94-176	63,93
Codec	commons-codec	18	1-18	None
Collections	commons-collections	4	25-28	1-24
Compress	commons-compress	47	1-47	None

续表 4-1

标识符	项目名称	缺陷数量	活跃缺陷编号	已废弃缺陷编号(*)
Csv	commons-csv	16	1-16	None
Gson	gson	18	1-18	None
JacksonCore	jackson-core	26	1-26	None
JacksonData bind	jackson-databind	112	1-112	None
JacksonXml	jackson-dataformat-xml	6	1-6	None
Jsoup	jsoup	93	1-93	None
JXPath	commons-jxpath	22	1-22	None
Lang	commons-lang	64	1,3-65	2
Math	commons-math	106	1-106	None
Mockito	mockito	38	1-38	None
Time	joda-time	26	1-20,22-27	21

综合考虑项目多样性，项目规模，缺陷数量，项目重要性与声誉等因素，选择以下 7 个项目开展对比实验：

(1) Chart(jfreechart):

JFreeChart 是一款开源的 Java 图表库，可生成丰富的交互式 and 静态图表，支持跨平台（JVM）环境，使用 LGPL 许可证进行分发。该库支持多种图表类型，包括 X-Y（折线、样条、散点）、饼图、甘特图、条形图（水平/垂直、堆叠/独立）以及特定用途的仪表图和气泡图，并提供自动缩放、注释和图例功能。

(2) Cli(commons-cli):

Apache Commons CLI 提供了解析命令行选项的统一 API，可帮助程序定义、解析并查询命令行参数，简化工具的接口设计与维护。其处理流程分为定义、解析和查询三阶段，并能自动生成详细的帮助信息，适用于各种命令行工具。

(3) Compress(commons-compress):

Apache Commons Compress 定义了对多种压缩格式（如 ar、cpio、tar、zip、gzip、bzip2、7z、LZMA、XZ、Snappy、LZ4、Brotli、Zstandard 等）的统一操作 API，支持流式与文件集操作。该组件源自多种开源项目，实现了丰富的压缩算法和格式支持，便于在 Java 应用中灵活集成压缩与解压功能。

(4) Csv(commons-csv):

Apache Commons CSV 是一个用于读取与写入各种 CSV 变体文件的 Java 库，提供简单易用的接口，支持多种分隔符、引用字符和换行处理。

(5) Gson(gson):

Gson(Google Gson)是谷歌开源的 Java 对象与 JSON 相互转换库,支持将 Java 对象序列化为 JSON 并反序列化回 Java 对象,采用 Apache License 2.0。基于反射机制, Gson 无需修改源代码即可处理集合、泛型和嵌套类等复杂类型,并提供自定义序列化 / 反序列化器以满足特殊需求。

(6) JXPath(common-jxpath):

Apache Commons JXPath 提供了一种通过 XPath 表达式遍历 Java 对象图的机制,将 XML 风格的路径查询引入 Java Bean、集合和 Map 等数据结构,简化了复杂对象导航。

(7) Math(common-math):

Apache Commons Math 是一套轻量级、自包含的数学与统计组件库,涵盖线性代数、优化、数值积分、随机数生成、概率分布和微分方程等常见数学问题。

这些项目的领域多样性强,功能覆盖广泛,项目成熟且较为稳定,行业广泛采用,而且部分项目具有较高的数学与算法要求,因此这些项目能够很好地测试自动化测试用例生成器于真实软件项目的表现。

4.1.2 对比基准

本文选择以下三个对比基准,包含最先进的 SBST 方法 EvoSuite 和两个最先进的基于大语言模型方法:ChatUniTest 和 HITS。

EvoSuite 是一个采用混合算法的针对 Java 程序的进化算法测试套件生成器,自动生成高覆盖率且带断言的 JUnit 测试。EvoSuite 是一个采用传统算法的基于搜索的自动测试用例生成工具^[20]。

ChatUniTest 使用焦点方法和通过规则提取的上下文作为输入,生成使用 LLM 的单元测试。它利用 JVM 的错误报告作为失败测试的输入,指示 LLM 修复这些测试^[9]。

HITS 通过将程序切片,并让大语言模型在切片上生成测试,缩小了分析范围,使 LLM 更容易覆盖每个切片中的更多行和分支。最终在实验中 HITS 在行和分支覆盖率得分方面显著优于当时使用 LLM 和典型 SBST 方法 EvoSuite 的测试用例生成方法^[16]。

4.1.3 评价指标

为了全面评估所提多 Agent 对抗测试用例生成方法的性能,本研究选取以下三类关键指标:Token 消耗、覆盖率和缺陷检出率。这三项指标分别从成本、充分性

和有效性三个维度对测试用例质量与生成效率进行衡量。

(1) Token 消耗

Token 消耗直接反映了方法的资源开销与经济成本。大语言模型的使用往往按 Token 付费，通过度量 Token 消耗，可评估在同等质量要求下，不同策略的成本效率差异。

本文将对比 HITS 与本文所提测试用例生成方法所用的 Token 消耗数量。

(2) 覆盖率

覆盖率是衡量测试充分性的经典标准，能够反映测试用例对代码路径的探索深度。较高的覆盖率意味着测试用例能够触发更多代码分支，提升对潜在缺陷的暴露几率。

(3) 缺陷检出率

缺陷检出率直接衡量测试用例对真实缺陷的定位能力与准确性，是验证测试生成方法有效性的核心指标。

单纯的覆盖率并不能完全代表测试用例的有效性，只有能够准确触发实际缺陷的测试用例才具有真正的缺陷揭露价值。通过计算缺陷检出率，可定量对比不同生成策略在发现真实缺陷方面的优势。

缺陷检出率定义为：对于测试用例集 T ，分别在修复版本和缺陷版本上运行。分别得到修复版本在测试用例集 T 上失败的用例集合 T_{fix} 、 T_{bug} 若 $T_{bug} = T_{fix}$ ，则说测试用例集 T 无法检出缺陷。若 $T_{bug} \neq T_{fix}$ 或测试用例 T 能够在修复版本上正常运行且在缺陷版本上出现编译或运行时错误，则说测试用例集 T 能够检出缺陷。对于每个项目，缺陷检出率等于每个项目检出的缺陷数量除以该项目的总缺陷数量。

4.1.4 环境配置

为了与 Defects4J V2.0.1 兼容，生成的测试用例使用 Java8 与 Junit4, Mockito4.11。DeepseekV3-0324, temperature=0，因其为代码生成项目。目标覆盖率 95%，目标变异得分 70%。

EvoSuite 使用 Defects4J 中的版本，最大搜索时间设置为 100 秒。

对于 ChatUniTest 与 HITS 作了必要的修改，使其与 Defects4J 框架兼容。例如修改其提示词中的“Junit5”为“Junit4”。

4.2 结果与分析

4.2.1 缺陷检出率

表 4-2 缺陷检出率结果

	EvoSuite	ChatUniTest	HITS	本文方法
Chart	50.00%	57.69%	<u>65.38%</u>	73.08%
Cli	46.15%	30.77%	<u>71.79%</u>	76.92%
Compress	42.55%	25.53%	48.94%	<u>46.81%</u>
Csv	<u>62.50%</u>	25.00%	68.75%	56.25%
Gson	44.44%	38.89%	66.67%	<u>61.11%</u>
JXPath	<u>45.45%</u>	27.27%	40.91%	72.73%
Math	34.91%	37.74%	59.43%	59.43%
项目平均值	42.34%	35.04%	<u>59.49%</u>	62.04%
总检出率	46.57%	34.70%	<u>60.27%</u>	63.76%

总的来说，本文提出的方法在大部分项目表现优异，具有最高的总检出率，相较于 HITS 提升了约 5%的缺陷检出率。另外，分项目来看，本文在 4 个项目上领先，在 2 个项目上表现次优，在一个项目上排名第三。

可以看到，传统的 SBST 方法 EvoSuite 表现较为稳定，并且在较为简单的 Csv 项目中表现出色。而本文的方法与 HITS 在较复杂的项目或数理逻辑较多的项目上，例如 Chart 和 Math 上表现出色，因为 HITS 通过切片的方式降低了方法的复杂程度，而本文方法可以通过存活变异体的提示来帮助大语言模型创建测试或修复逻辑。而在较为简单的项目，例如 Csv 和 Gson 上，本文方法表现反而略有不佳，而在 Compress 项目中，三个项目表现相当接近。在 JXPath 项目上，本文方法击败了 HITS，JXPath 项目包含复杂的数据结构，HITS 较难在其上生成切片与测试。

4.2.2 覆盖率

表 4-3 与表 4-4 为各个方法行覆盖率与分支覆盖率的结果。

在行覆盖率上，EvoSuite 在行覆盖率上整体领先，平均达 74.73%，得益于其进化搜索直接以覆盖率为优化目标。HITS 通过切片降低大模型分析难度，也获得了较高的行覆盖（71.35%），在 Cli 和 Compress 项目上超过了 EvoSuite。本文方法平均行覆盖为 67.86%，低于 EvoSuite 与 HITS。主要原因在于本文的多 Agent 对抗框

架更侧重于提高缺陷检出和分支覆盖，对“常规”分支和常见代码路径的覆盖提升不如专门优化覆盖率的搜索式或切片式方法。

表 4-3 行覆盖率结果

	EvoSuite	ChatUniTest	HITS	本文方法
Chart	80.46%	70.80%	78.45%	73.21%
Cli	88.24%	59.85%	93.59%	88.85%
Compress	48.10%	42.70%	64.23%	56.86%
Csv	95.13%	44.70%	91.48%	90.96%
Gson	48.58%	24.21%	<u>47.79%</u>	34.23%
JXPath	78.69%	41.68%	44.75%	<u>54.08%</u>
Math	83.92%	38.89%	<u>79.19%</u>	76.84%
总覆盖率	74.20%	42.86%	<u>71.13%</u>	67.38%
项目平均值	74.73%	46.12%	<u>71.35%</u>	67.86%

表 4-4 分支覆盖率结果

	EvoSuite	ChatUniTest	HITS	本文方法
Chart	74.14%	66.51%	57.32%	<u>71.65%</u>
Cli	79.08%	53.39%	87.45%	88.84%
Compress	50.77%	44.50%	52.62%	54.57%
Csv	91.75%	43.42%	84.87%	<u>87.43%</u>
Gson	45.44%	24.52%	46.08%	33.12%
JXPath	72.24%	37.92%	37.49%	<u>53.01%</u>
Math	80.45%	30.86%	66.75%	<u>69.53%</u>
总覆盖率	71.05%	39.17%	60.29%	<u>63.85%</u>
项目平均值	70.55%	43.02%	61.80%	<u>65.45%</u>

在分支覆盖率指标上，EvoSuite 依然表现最优（70.55%），但本文方法（65.45%）已超过 HITS（61.80%），位居第二。特别是在 Chart、Cli、Compress、JXPath 这些项目上，本文方法均优于 HITS，说明多 Agent 对抗策略在难触达分支或逻辑复杂处能更有效地引导 LLM 生成针对性测试用例。ChatUniTest 在所有项目上分支覆盖率最低（43.02%），其流程对分支探索能力的不足。HITS 在分支覆盖率上能力

下降显著，可能的原因是大语言模型在切片时可能会忽略较短分支，导致在后面的生成中难以生成覆盖该分支的测试。

另外，Evosuite 在 Math 项目上的表现：其在覆盖率上领先，却在缺陷检测率上落后于 LLM 方法，再一次表明了覆盖率与缺陷检出率之间复杂的关系。还表明大语言模型中的其他领域知识（例如数学知识）能够帮助其生成更能检测出可能缺陷的测试。而 SBST 完全不了解程序的背景或意图，导致其在可能出现的缺陷的发现上落后。

4. 2. 3 Token 消耗

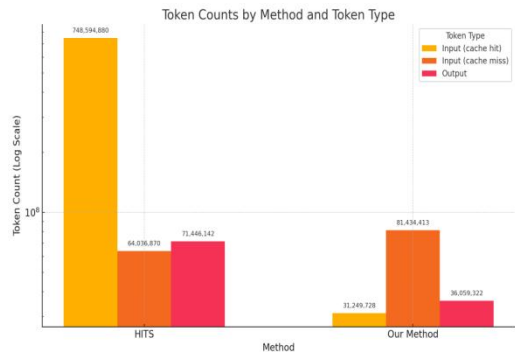


图 4-1 HITS 与本方法的 token 消耗对比

图 4-1 比较了 HITS 与本文方法的 Token 消耗(经过所有 7 个项目生成后的 token 消耗)。尽管本文方法的总消耗低于 HITS，其缓存命中率却显著偏低。对此，有以下推测：本文方法在测试用例生成时采用“按项目逐一生成、按缺陷逐一生成”处理，而 HITS 则以流水线方式：先对所有项目执行第一步，再统一执行第二步，从而显著提高了缓存利用率。另外 HITS 在提示词中会将整个类提供给大模型，因此其输入 Token 数也显著的高于本文的方法。

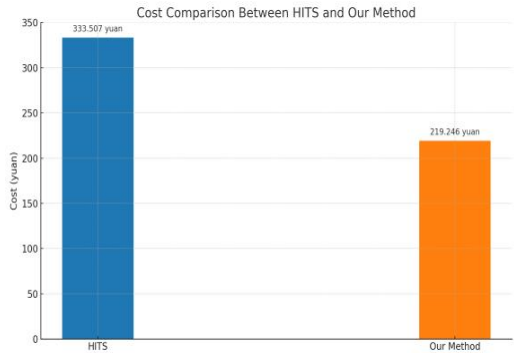


图 4-2 HITS 与本方法的成本对比

图 4-2 比较了 HITS 与本文方法的成本（以人民币计算）。本文方法节省了约 34% 的成本。

需要注意的是，Token 消耗与 deepseek 使用的 tokenizer 有关，并且可能受到当时网络连接情况的影响，因此不同的模型甚至不同的模型提供者均有可能影响 token 消耗。成本对比方面，由于平台计价策略的设置，平台计算成本会因时段的不同而产生区别，而不同模型的输入输出定价也可能不同，因此不同的模型与不同的模型提供者均有可能影响成本。

4.2.4 案例研究

本小节以 Chart 项目中第 13 号缺陷为例，演示如何通过变异测试生成能够暴露该缺陷的测试用例。针对该缺陷，只有本方法成功通过对变异体的分析生成的测试用例将其检出。

```
The Patch of Chart13:  
h[3] = h[2];  
if (this.rightBlock != null) {  
    RectangleConstraint c4 = new RectangleConstraint(0.0,  
+ new Range(0.0, Math.max(constraint.getWidth() - w[2],  
0.0)),  
- new Range(0.0, constraint.getWidth() - w[2]),  
    LengthConstraintType.RANGE, h[2], null,  
    LengthConstraintType.FIXED);  
    Size2D size = this.rightBlock.arrange(g2, c4);
```

图 4-3 修复缺陷的更新

```
{  
  "id": 8,  
  "line": 455,  
  "precode": "new Range(0.0, Math.max(constraint.getWidth() - w[2], 0.0))",  
  "filepath": "...",  
  "aftercode": "new Range(0.0, Math.min(constraint.getWidth() - w[2], 0.0))",  
},  
{  
  "id": 9,  
  "line": 455,  
  "precode": "new Range(0.0, Math.max(constraint.getWidth() - w[2], 0.0))",  
  "filepath": "...",  
  "aftercode": "new Range(0.0, constraint.getWidth() - w[2])",  
},
```

图 4-4 生成的变异体

修复该缺陷的更新如图 4-3 所示，在限制宽度 (`constraint.getWidth()`) 小于实际宽度 (`w[2]`) 的情况出现时，构造 `Range` 的第二个参数会为负数，进而导致程序运行出现 `IllegalArgumentException` 异常。

初始变异体生成部分生成的变异体见图 4-4。在初始变异体生成时，已经有 2 个变异体能够模拟该缺陷：其中 `id=9` 的变异体与缺陷版本完全一样，`id=8` 的变异体也可以模拟该缺陷，其也会在限制宽度小于实际宽度时出现异常。

下文分析测试用例生成 `Agent` 如何尝试杀死上文中的变异体并最终能够检测出该缺陷。

```
@Test
public void testArrangeFF_DetectNegativeWidthRangeMutant() throws Exception {
    // Setup
    BorderArrangement arrangement = new BorderArrangement();
    BlockContainer container = mock(BlockContainer.class);
    Graphics2D g2 = mock(Graphics2D.class);

    // Create a scenario where constraint.getWidth() - w[2] will be negative
    RectangleConstraint constraint = new RectangleConstraint(100.0, 200.0);

    // Mock blocks to control their behavior
    Block leftBlock = mock(Block.class);
    Block rightBlock = mock(Block.class);

    // Set the left block to return a width larger than the constraint
    Size2D leftBlockSize = new Size2D(150.0, 50.0); // width > constraint width
    when(leftBlock.arrange(any(Graphics2D.class), any(RectangleConstraint.class)))
        .thenReturn(leftBlockSize);

    // Set the right block to verify it gets proper constraints
    Size2D rightBlockSize = new Size2D(0.0, 50.0);
    when(rightBlock.arrange(any(Graphics2D.class), any(RectangleConstraint.class)))
        .thenReturn(rightBlockSize);

    // Add blocks to the arrangement
    arrangement.add(leftBlock, RectangleEdge.LEFT);
    arrangement.add(rightBlock, RectangleEdge.RIGHT);

    // Use reflection to access protected arrangeFF method
    Method arrangeFF = BorderArrangement.class.getDeclaredMethod(
        "arrangeFF",
        BlockContainer.class,
        Graphics2D.class,
        RectangleConstraint.class
    );
    arrangeFF.setAccessible(true);

    // Invoke arrangeFF and capture the result
    Size2D result = (Size2D) arrangeFF.invoke(arrangement, container, g2, constraint);

    // Verify the arrangeFF invocation completed without exception
    assertEquals(100.0, result.getWidth(), 0.0001);
    assertEquals(200.0, result.getHeight(), 0.0001);

    // Verify the right block was arranged with a valid (non-negative) width range
    verify(rightBlock).arrange(eq(g2), any(RectangleConstraint.class));
}
```

图 4-5 生成的能杀死变异体的测试用例

其中能杀死上面两个变异体的用例如图 4-5 所示，注意它的注释显示了其如何去设置条件去使得 `constraint.getWidth()-w[2]<0`。另外，该测试用例熟练地运用了 `Reflection` 与 `Mockito` 等测试必要的方法和技巧。该测试用例能顺利分辨出修复

后的版本与缺陷版本，缺陷版本在运行该测试用例的时候触发了异常退出，而修复版本成功运行了该用例并通过所有的断言和验证。

这个用例体现了本方法的优越性，若仅采用基于覆盖率的反馈，只要满足 `rightBlock!=null` 就会将这一行覆盖，然而潜藏的缺陷却出现在这种“隐形分支”中，基于覆盖率的反馈无法将其发现，依靠变异测试，才能引导测试用例生成 Agent 生成能杀死该变异体的测试用例，进而发现该缺陷。另外，这也体现了大模型生成变异体的优越性，因为其没有变异算子的限制，才能生成这两个变异，传统方法无法生成这两个变异体，因为其中一个是直接拿掉方法调用，另一个是把 `max` 改为 `min`。无法保证通过编译，因此无法通过变异算子来生成这两个变异体。然而大模型运用其代码知识，生成了这两个对人类来说很合理的变异体。

另外，对本文方法未成功检出缺陷的一些实验中，发现其过程中往往出现两种情况。一是过早地满足了终止条件，此时可能变异体生成器并未充分地对焦点方法进行变异体生成。二是测试用例生成器很难杀死变异体，导致变异得分始终较低。这两种情况背后的原因以及可能的改进方法在 5.2 节中进行说明。

4.3 本章小结

本章节主要介绍了对本文方法的实验的实验设置以及对比基准。并对实验结果进行了分析。实验结果表明：本文方法在缺陷检测率上领先于先前最先进的 LLM 方法 HITS 达 4.29%，优于最先进的 SBST 方法 EvoSuite 达 46.5%。并且本文的 Token 消耗量少于 HITS，成本相较 HITS 减少了约 30%。覆盖率方面，本文在行覆盖率上略逊于 Evosuite 与 HITS，在分支覆盖率上则超过 HITS，仅略逊于 Evosuite。另外，在本章节的最后，以 Chart 项目的第 13 号缺陷为例，进行了案例分析，介绍了本文方法的一般流程以及其为何能够检出其他方法无法检出的缺陷。

第五章 全文总结

5.1 主要结论

本文主要提出了一个新的基于大模型多 Agent 对抗的测试用例生成方法，并于 Defects4J 数据集上作了测试，测试结果表明，该方法在自动用例生成的缺陷检出率优于先前最先进的 LLM 方法 HITS 4.29%，优于最先进的 SBST 方法 EvoSuite 46.5%。在覆盖率方面，本文方法在行覆盖率上略逊于最先进的 LLM 方法与 SBST 方法，在分支覆盖率上，本文方法表现优异，只略逊于 Evosuite。在 Token 消耗方面，本文方法的成本消耗相较 HITS 减少了约 30%。以上结果说明该反馈循环的策略成功达成了提高测试用例缺陷检出率、提高覆盖率的效果。

5.2 本文的不足与研究展望

在本章节，将会介绍本文的研究过程中的有不足和问题，以及对未来该领域研究发展的展望。

5.2.1 等价变异体识别

本文曾在 2.2 节提到，等价变异体的识别是一个不可判定问题，但是有许多研究者提出了许多方法针对这一问题，研究者也对大语言模型能否运用其语义理解能力，来解决等价变异体识别问题。Ma W 等人[68]首先尝试用 ChatGPT 对小规模变异体对进行零样本等价性判断，取得了初步成果。文献[69]探索了多种使用大语言模型进行等价变异体判断的方法，结果表明：除 CodeBERT 外，几乎所有 LLM 方法在 Macro F1 值上平均领先编译器方法 75.2%、领先机器学习方法 19.1%、领先 ASTNN 等树模型 12.8%，而且在准确率(precision)和召回率(recall)两个指标上效果均领先。而在 LLM 方法中，使用微调的代码嵌入策略(fine-tuned code embedding)效果最好，零样本和少样本的提示策略效果较差。而本文的方法中，减少等价变异体的生成和对等价变异体的判断依赖于提示策略，而等价变异体识别的能力可能影响了本方法性能。因此，未来可以考虑添加一个于等价变异体任务上微调的小模型，

用于识别等价变异体。

5.2.2 存活变异体辅助测试用例生成的其他方法

针对变异测试，文献[70]提出了通过迭代大语言模型驱动的科学调试(scientific debugging)进行变异测试。在这种方法中，大语言模型会形成关于如何杀死特定变异体的假设，然后迭代地生成和优化测试，直到成功区分变异体。

然而，采用这种方法，相较于基线方法（只询问一次），虽然生成成功区分变异体的测试的概率增加了，但是文献数据表明，Token 消耗为基线方法的十倍[70]。而且这种方法经过多次失败可能会把该变异体标记为等价，但很多时候这个被标记为等价的变异体并非等价变异体。考虑到成本与效果的平衡，并未于本文的方法中使用类似方法。但是提升成功生成杀死该存活变异体的测试用例的概率仍是未来可以提升的方向。

5.2.3 未来自动生成测试用例方法的展望

针对单个焦点方法的测试生成，需要多轮大模型调用，带来显著的时间成本与Token 消耗。在大规模项目级测试场景下，若不加以优化，资源开销会快速攀升。例如，在本文中可将变异体生成与变异测试并行执行，或对 LLM API 请求进行批量并发调用，并结合本地缓存等手段，进一步压缩总耗时。而且随着大语言模型能力的增强，其执行复杂任务的能力也不断增强。因此未来的自动生成测试用例方法可能往项目级测试用例生成发展，并为大语言模型 Agent 提供更灵活的空间和更高的自主性^[9]。为了避免一次性输入过长导致达到 Token 上限和上下文稀释，可结合检索增强生成（Retrieval-Augmented Generation, RAG）技术，动态检索项目文档及代码，降低整体资源消耗。

另外，本方法并非仅限于 Java 语言的测试用例生成，未来可考虑实现 Python 或其他语言版本。

结合方法特点，本方法更适合于回归测试场景下的增量测试用例生成，而非新项目的全新测试开发，因为本方法也如同之前大部分的自动测试用例生成方法（SBST，以及大部分基于 LLM 的方法），假设输入的待测代码为正确代码。未来可探索基于需求规约或项目文档的自动化触发机制，实现文档驱动测试。但是大语言模型的幻觉和出错问题在这种条件下很难得到有效的解决或控制，而且难以验证大语言模型是否生成正确的测试。未来随着大语言模型能力的增强，其生成的代码量越来越多并且其越来越广泛地应用在生产中，对于代码正确性的检查和验证就越关

键，软件测试在未来仍将会是研究的重点。

参 考 文 献

- [1] Shore J, Warden S. The art of agile development[M]. " O'Reilly Media, Inc.", 2021.
- [2] Siddiqui S. Learning Test-Driven Development[M]. " O'Reilly Media, Inc.", 2021.
- [3] Beller M, Gousios G, Panichella A, et al. When, how, and why developers (do not) test in their IDEs[C]//Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. 2015: 179-190.
- [4] Beller M, Gousios G, Zaidman A. How (much) do developers test?[C]//2015 IEEE/ACM 37th IEEE International Conference on Software Engineering. IEEE, 2015, 2: 559-562.
- [5] Brandt C. The Interplay Between Automatic Generation and Human Exploration in Developer-Centric Test Amplification[J].2022
- [6] Jain K, Goues C L. TestForge: Feedback-Driven, Agentic Test Suite Generation[J]. arXiv preprint arXiv:2503.14713, 2025.
- [7] Fraser G, Arcuri A. EvoSuite: automatic test suite generation for object-oriented software[C]//Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering. 2011: 416-419.
- [8] Lukasczyk S, Fraser G. Pynguin: Automated unit test generation for python[C]//Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings. 2022: 168-172.
- [9] Chen Y, Hu Z, Zhi C, et al. Chatunitest: A framework for llm-based test generation[C]//Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. 2024: 572-576.
- [10] Deljouyi A, Koohestani R, Izadi M, et al. Leveraging large language models for enhancing the understandability of generated unit tests[J]. arXiv preprint arXiv:2408.11710, 2024.
- [11] Liu J, Wang K, Chen Y, et al. Large language model-based agents for software engineering: A survey[J]. arXiv preprint arXiv:2409.02977, 2024.
- [12] Goodfellow I, Pouget-Abadie J, Mirza M, et al. Generative adversarial nets[J]. Advances in neural information processing systems, 2014, 27.
- [13] Ling Y, Jiang X, Kim Y. MALLM-GAN: Multi-Agent Large Language Model as Generative Adversarial Network for Synthesizing Tabular Data[J]. arXiv preprint arXiv:2406.10521, 2024.
- [14] Just R, Jalali D, Inozemtseva L, et al. Are mutants a valid substitute for real faults in software testing?[C]//Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. 2014: 654-665.
- [15] Dakhel A M, Nikanjam A, Majdinasab V, et al. Effective test generation using pre-trained large language models and mutation testing[J]. Information and Software Technology, 2024, 171: 107468.
- [16] Wang Z, Liu K, Li G, et al. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing[C]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 1258-1268.
- [17] Chen M, Tworek J, Jun H, et al. Evaluating large language models trained on code[J]. arXiv preprint arXiv:2107.03374, 2021.
- [18] Hu Y, Ahmed U Z, Mechtaev S, et al. Re-factoring based program repair applied to programming assignments[C]//2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2019: 388-398.
- [19] Coles H, Laurent T, Henard C, et al. Pit: a practical mutation testing tool for java[C]//Proceedings of the 25th international symposium on software testing and analysis. 2016: 449-452.
- [20] Just R, Jalali D, Ernst M D. Defects4J: A database of existing faults to enable controlled testing studies for Java programs[C]//Proceedings of the 2014 international symposium on softwa

re testing and analysis. 2014: 437-440.

[21] 中华人民共和国国家质量监督检验检疫总局, 中国国家标准化管理委员会. 信息技术 软件工程术语: GB/T 11457-2006[S]. 北京: 中国标准出版社, 2006.

[22] Pacheco C, Ernst M D. Randoop: feedback-directed random testing for Java[C]//Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion. 2007: 815-816.

[23] Fioraldi A, Maier D, Eißfeldt H, et al. {AFL++}: Combining incremental steps of fuzzing research[C]//14th USENIX workshop on offensive technologies (WOOT 20). 2020.

[24] Claessen K, Hughes J. QuickCheck: a lightweight tool for random testing of Haskell programs[C]//Proceedings of the fifth ACM SIGPLAN international conference on Functional programming. 2000: 268-279.

[25] Boyapati C, Khurshid S, Marinov D. Korat: Automated testing based on Java predicates[J]. ACM SIGSOFT Software Engineering Notes, 2002, 27(4): 123-133.

[26] MacIver D R, Hatfield-Dodds Z. Hypothesis: A new approach to property-based testing[J]. Journal of Open Source Software, 2019, 4(43): 1891.

[27] Tillmann N, De Halleux J. Pex—white box test generation for .net[C]//International conference on tests and proofs. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008: 134-153.

[28] Cadar C, Dunbar D, Engler D R. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs[C]//OSDI. 2008, 8: 209-224.

[29] Choi J, Jang J, Han C, et al. Grey-box concolic testing on binary code[C]//2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019: 736-747.

[30] Tufano M, Drain D, Svyatkovskiy A, et al. Unit test case generation with transformers and focal context[J]. arXiv preprint arXiv:2009.05617, 2020.

[31] Alagarsamy S, Tantithamthavorn C, Aleti A. A3test: Assertion-augmented automated test case generation[J]. Information and Software Technology, 2024, 176: 107565.

[32] Villmow J, Depoix J, Ulges A. CONTEST: A unit test completion benchmark featuring context[C]//Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog 2021). 2021: 17-25.

[33] Nie P, Banerjee R, Li J J, et al. Learning deep semantics for test completion[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 2111-2123.

[34] Rao N, Jain K, Alon U, et al. CAT-LM training language models on aligned code and tests[C]//2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023: 409-420.

[35] Siddiqua M L, Santos J C, Tanvir R H, et al. An empirical study of using large language models for unit test generation[J]. arXiv preprint arXiv:2305.00418, 2023.

[36] Schäfer M, Nadi S, Eghbali A, et al. An empirical evaluation of using large language models for automated unit test generation[J]. IEEE Transactions on Software Engineering, 2023, 50(1): 85-105.

[37] Lemieux C, Inala J P, Lahiri S K, et al. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 919-931.

[38] Ryan G, Jain S, Shang M, et al. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm[J]. Proceedings of the ACM on Software Engineering, 2024, 1(FSE): 951-971.

[39] Pizzorno J A, Berger E D. CoverUp: Coverage-Guided LLM-Based Test Generation[J]. arXiv preprint arXiv:2403.16218, 2024.

[40] Alshahwan N, Chheda J, Finogenova A, et al. Automated unit test improvement using large language models at meta[C]//Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. 2024: 185-196.

[41] Offutt A J, Pan J. Automatically detecting equivalent mutants and infeasible paths[J]. Software testing, verification and reliability, 1997, 7(3): 165-192.

[42] 陈翔, 顾庆. 变异测试: 原理, 优化和应用[J]. 计算机科学与探索, 2012, 6(12): 1057-1075.

- [43] DeMillo R A, Lipton R J, Sayward F G. Hints on test data selection: Help for the practicing programmer[J]. Computer, 1978, 11(4): 34-41.
- [44] DeMillo R A, Mathur A P. On the use of software artifacts to evaluate the effectiveness of mutation analysis for detecting errors in production software[R]. Technical report, Software Engineering Research Center, Purdue University, SERC-TR-92-P, 1991.
- [45] Offutt A J. Investigations of the software testing coupling effect[J]. ACM Transactions on Software Engineering and Methodology (TOSEM), 1992, 1(1): 5-20.
- [46] Schuler D, Zeller A. Javalanche: Efficient mutation testing for Java[C]//Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering. 2009: 297-298.
- [47] Just R, Schweiggert F, Kapfhammer G M. MAJOR: An efficient and extensible tool for mutation analysis in a Java compiler[C]//2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011). IEEE, 2011: 612-615.
- [48] Delamaro M E, Maldonado J C, Vincenzi A M R. Proteum/IM 2.0: An integrated mutation testing environment[J]. Mutation testing for the new century, 2001: 91-101.
- [49] Jia Y, Harman M. MILU: A customizable, runtime-optimized higher order mutation testing tool for the full C language[C]//Testing: Academic & Industrial Conference-Practice and Research Techniques (taic part 2008). IEEE, 2008: 94-98.
- [50] Jia Y, Harman M. Higher order mutation testing[J]. Information and Software Technology, 2009, 51(10): 1379-1393.
- [51] Wang B, Xiong Y, Shi Y, et al. Faster mutation analysis via equivalence modulo states[C]//Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis. 2017: 295-306.
- [52] Wang B, Lu S, Xiong Y, et al. Faster mutation analysis with fewer processes and smaller overheads[C]//2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2021: 381-393.
- [53] Derezinska A, Halas K. Experimental evaluation of mutation testing approaches to python programs[C]//2014 IEEE seventh international conference on software testing, verification and validation workshops. IEEE, 2014: 156-164.
- [54] Ma L, Zhang F, Sun J, et al. Deepmutation: Mutation testing of deep learning systems[C]//2018 IEEE 29th international symposium on software reliability engineering (ISSRE). IEEE, 2018: 100-111.
- [55] Tian Z, Chen J, Zhu Q, et al. Learning to construct better mutation faults[C]//Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. 2022: 1-13.
- [56] Degiovanni R, Papadakis M. µbert: Mutation testing using pre-trained language models[C]//2022 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW). IEEE, 2022: 160-169.
- [57] Tip F, Bell J, Schäfer M. Llmorpheus: Mutation testing using large language models[J]. IEEE Transactions on Software Engineering, 2025.
- [58] WANG B O, CHEN M, LIN Y, et al. On the Use of Large Language Models in Mutation Testing[J]. arXiv preprint arXiv:2406.09843, 2024.
- [59] Wu Y, Li Z, Zhang J M, et al. Condefects: A complementary dataset to address the data leakage concern for llm-based fault localization and program repair[C]//Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering. 2024: 642-646.
- [60] Garg A, Degiovanni R, Papadakis M, et al. On the Coupling between Vulnerabilities and LLM-generated Mutants: A Study on Vul4J dataset[C]//2024 IEEE Conference on Software Testing, Verification and Validation (ICST). IEEE, 2024: 305-316.
- [61] Jain N, Han K, Gu A, et al. Livecodebench: Holistic and contamination free evaluation of large language models for code[J]. arXiv preprint arXiv:2403.07974, 2024.
- [62] Liu A, Feng B, Xue B, et al. Deepseek-v3 technical report[J]. arXiv preprint arXiv:2412.1

9437, 2024.

[63] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models[J]. Advances in neural information processing systems, 2022, 35: 24824-24837.

[64] Li J, Li G, Li Y, et al. Structured chain-of-thought prompting for code generation[J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(2): 1-23.

[65] Marvin G, Hellen N, Jjingo D, et al. Prompt engineering in large language models[C]//International conference on data intelligence and cognitive informatics. Singapore: Springer Nature Singapore, 2023: 387-402.

[66] Kojima T, Gu S S, Reid M, et al. Large language models are zero-shot reasoners[J]. Advances in neural information processing systems, 2022, 35: 22199-22213.

[67] Seo H, Sadowski C, Elbaum S, et al. Programmers' build errors: a case study (at google) [C]//Proceedings of the 36th International Conference on Software Engineering. 2014: 724-734.

[68] Ma W, Liu S, Wang W, et al. The scope of chatgpt in software engineering: A thorough investigation[J]. arXiv preprint arXiv:2305.12138, 2023.

[69] Tian Z, Shu H, Wang D, et al. Large Language Models for Equivalent Mutant Detection: How Far Are We?[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2024: 1733-1745.

[70] Straubinger P, Kreis M, Lukasczyk S, et al. Mutation Testing via Iterative Large Language Model-Driven Scientific Debugging[C]//2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW). IEEE, 2025: 358-367.

完整提示词模板（附录 1）

A:Agent T 初次生成:

Instruction:

You are an expert Java developer and software tester. Your task is to generate full JUnit test methods for a given Java method inside a Java class. Follow these steps to ensure comprehensive and effective test coverage:

1. **Analyze the Java Method**:

- Identify the method's parameters and return type.
- Understand the functionality and purpose of the method.

2. **Design Test Cases**:

- For each method, design test cases that cover typical usage, edge cases, and potential error conditions.
- Ensure that all possible execution paths are tested.

3. **Implement the Test Method**:

- Make Sure your variables are declared inside your test Method.
- Write a test method annotated with `@Test` for each test case.
- Use assertions to verify the expected outcomes.
- Finish the code on your own including the assertions, you are the project owner instead of me.

Example:

Input Java Method (add):

```

```
public class Calculator{
...
public int add(int a, int b) {{ return a + b; }}
...
}}
```

```

Generated JUnit Test Method (testAdd_TypicalValues):

```

@Test

```
public void testAdd_TypicalValues()
{
 Calculator calculator = new Calculator();
 assertEquals(5, calculator.add(2, 3));
 assertEquals(-1, calculator.add(-2, 1));
 assertEquals(0, calculator.add(0, 0)); } }
```

```

Your Task:

Given the following Java method, generate a complete JUnit test method that thoroughly tests the method. Utilize your reasoning ability to ensure that all possible scenarios and edge cases are considered.

_Input Java Method ({method_name}):

method_body:

'''

{method_code}

'''

Other Class Variables:

{class_variables}

Other Methods in the Class(no method body showed):

{method_info}

_Constructors of the class object:

{Constructors}

Guidelines:

- ****Annotations****: Use @Test to annotate each test method.
- ****Assertions****: Use appropriate assertions to validate expected outcomes.
- ****Completeness****: You should complete every assertions on your own, I would not add anything to your code. Make sure your test is as complete as you can.
- ****Exception Handling****: Ensure that methods throwing exceptions are properly tested.
- ****Thinking Step by Step****:
 - Begin by thoroughly understanding the structure and functionality of the provided Java method. Think critically about different scenarios and edge cases that the method should handle. Organize the test cases logically within the test method to maintain clarity and readability.
- ****Version Of Java and Junit****:
 - Use Version 1.8 of Java and Junit 4, you need to be aware of this.(Do not use Junit 5's feature like assertThrows,use ExpectedException instead)
 - Still need to use @Test to mark your respond.
- ****Mockito****:
 - Mockito is statically imported , so use "mock" instead of "Mockito.mock", If you really doesn't know the behavior of other class(not the tested class), you can mock it instead.
 - Do not abuse using Mockito, because wrongly using Mockito might lead to the test method hanging or timeout.
- _Note_:
- I use triple backticks to mark the code sessions.
- Do not give me the skeleton, give me the full code and everything should be implemented inside the code.

致 谢

首先我要感谢顾小东老师，我要感谢他对我鼓励和我研究、实验和论文的指导，这些指导和鼓励帮助我克服了研究中的种种困难。我还要感谢沈备军老师、林雅岚学姐、王光展学长和石雨凌学长在研究和写作过程中提供的建议和帮助。还要感谢家人与朋友在大学四年间给予的关怀和鼓励。以及上海交通大学为我提供良好的学习条件。

LARGE LANGUAGE MODEL BASED MULTI-AGENT ADVERSARIAL TEST CASE GENERATION

Software testing is a cornerstone of the software development lifecycle, tasked with verifying that programs meet their functional, performance, and security requirements. Unit testing—verifying individual methods or functions in isolation—constitutes the first line of defense against defects, yet its manual creation demands considerable developer effort and is fraught with potential oversights. While search based tools such as EvoSuite and Pygoblin have demonstrated the ability to systematically optimize for coverage metrics, the resulting test suites often fail to align with human conventions in naming, structure, and commentary, which impedes long term maintainability. On the other hand, large language models (LLMs) are capable of generating code fragments that resemble human-authored tests, complete with descriptive identifiers and contextual comments, but they commonly underperform in corner-case exploration and fall short of achieving high coverage without further guidance or more information. Filling in the gap between automated coverage optimization and high readability and understandability remains an open challenge.

A variety of hybrid strategies have sought to marry the strengths of search algorithms with the generative prowess of LLMs. For instance, slicing-guided paradigms—exemplified by HITS—narrow the scope of generation by feeding method-localized code slices into the LLM, resulting in focused but sometimes fragmented test cases. Other methodologies invest coverage feedback in iterative loops: CoverUP, for example, reruns LLM prompts with uncovered code segments flagged, while MuTAP integrates mutation testing on top of LLM outputs to simulate defect scenarios. Yet none of these approaches fully exploit an adversarial interplay between test-generation and mutant-synthesis to drive mutual refinement. This thesis addresses that gap by introducing a fully multi-agent adversarial methodology that leverages both mutation signals to evolve robust, maintainable and bug-detectable JUnit test suites for real-world Java programs.

Mutation testing itself has long been recognized as a fault-based technique for

evaluating test suite quality. By systematically introducing small syntactic changes — such as replacing operators, modifying constant values, or altering control-flow statements — into the program under test, one produces a collection of “mutants” that mimic common developer mistakes. A high quality test suite should kill these mutants by detecting behavioral deviations, and the resulting mutation score serves as a sensitive metric for the suite’s ability to capture subtle faults. Unlike coverage metrics that merely indicate which lines or branches have been exercised, mutation testing directly measures the suite’s efficacy in uncovering injected defects, offering deeper insight into test robustness. However, the brute-force generation and execution of mutants can be computationally expensive, and distinguishing equivalent mutants — those whose behavior remains semantically unchanged — poses an additional challenge. Integrating mutation feedback into an LLM-driven generation process, therefore, promises a more targeted and adversarial form of test evolution, focusing efforts on real behavioral weaknesses rather than superficial coverage gaps.

Recent research progress on mutation testing including deep learning methods and LLM-based methods on generating mutants. Deep learning has recast mutant generation as a translation task: for example, DeepMutation trains RNN encoder-decoders on large fault corpora, and LEAM uses AST-based encoding to capture both syntax and semantics. More recently, pre-trained LLMs have enabled zero- or few-shot synthesis — μ Bert predicts masked tokens for context-aware changes, LLMorpheus injects diverse errors into JavaScript, and models like GPT-4o yield mutants with greater AST diversity and stronger coupling to real vulnerabilities, detecting up to 29% more true faults than traditional tools.

At the heart of our approach lie two adversarial agents, each instantiated as an LLM-driven “actor” with distinct responsibilities. The Test-Case Generation Agent ingests essential context about a target method — its signature, import directives, class-level fields, and the signatures of neighboring methods — alongside exemplar test snippets, and its objective is to produce compilable, semantically meaningful JUnit test classes. In parallel, the Mutant Generation Agent receives the same contextual inputs and is tasked with creating high-challenge mutants: small but behavior-altering code transformations that compile successfully yet reveal weaknesses in the existing test suite.

Each mutant is encapsulated in a JSON object detailing the file path, line number, and pre/post-mutation code snippets.

The overall workflow unfolds in four sequential phases:

1.Initial Test-Case Generation and Repair.

The generation agent creates baseline test classes via engineered LLM prompts. An automated repair pipeline then addresses compilation defects: straightforward syntactic errors—such as unbalanced braces or missing imports—are rectified through rule - based heuristics, while persistent issues trigger up to ten rounds of targeted LLM - guided debugging. Repaired tests are consolidated into a unified suite with consistent package and naming conventions.

2.Initial Mutant Synthesis.

The mutant agent produces a diverse set of mutants for each method using adversarial prompts designed to avoid trivial, equivalent transformations. Non-compilable mutants are filtered out, ensuring that subsequent testing focuses on executable variants capable of revealing latent faults.

3.Mutation Testing and Coverage Measurement.

The assembled test suite is executed against each surviving mutant to ascertain kill rates, while coverage analysis tools record line and branch coverage. A customized mutation score metric excludes unreachable or non-compilable mutants from the denominator, providing a fair reflection of test efficacy.

4.Iterative Adversarial Refinement.

When mutation scores or coverage thresholds fall short of predefined targets, surviving mutants and uncovered code regions guide the next generation of test cases. The generation agent incorporates mutant-specific feedback—highlighting mutated lines and behavioral discrepancies—to craft focused tests, while the mutant agent spawns additional variants that target residual coverage gaps and previously surviving mutants. This adversarial cycle repeats up to four iterations or until convergence criteria are met, balancing thoroughness against computational cost.

We validate our approach on the Defects4J benchmark suite, comprising seven industrial-scale Java projects (Chart, Cli, Compress, Csv, Gson, JXPath, Math). The framework achieves a fault-detection rate that exceeds an LLM-only baseline (HITS) by

4.29% and outperforms the search-based EvoSuite by 46.5%. In terms of line coverage, our method attains an average of 67.86%, remaining competitive with both EvoSuite and slice-guided approaches. More notably, it secures an average branch coverage of 65.45%, surpassing HITS (61.80%) and approaching EvoSuite’s 70.55%, with particularly strong gains in complex projects such as Chart and JXPath. Detailed analysis in Tables 4-3 and 4-4 illustrates how adversarial cycles concentrate test efforts on hard-to-reach branches, uncovering edge-case behaviors like negative-width constraints that induce `IllegalArgumentException` errors in Chart. A comprehensive resource-consumption study (Section 4.2.3) reveals that most improvements accrue within the first three adversarial rounds, informing pragmatic termination strategies.

Beyond quantitative gains, qualitative case studies demonstrate that generated tests exhibit human-like readability, maintainable structure, and insightful commentary — attributes critical for real-world adoption. While the adversarial interplay incurs additional token and time costs relative to single-pass LLM generation, the improved defect detection and branch coverage justify the investment for high-assurance systems. Constraints on very large codebases — stemming from token-length limits and mutant volume — suggest promising avenues for future work, including slicing-guided agent orchestration, hybrid fitness criteria that blend coverage and mutation metrics, and advanced semantic analysis for equivalent-mutant detection.

Key Contributions:

A novel adversarial multi-agent paradigm that synergizes LLM-driven test generation with dynamic mutant synthesis, fostering continuous mutual refinement of test suites and mutant challenges.

A four-phase workflow — spanning initial generation and repair, mutant creation, mutation testing, and iterative adversarial loops — that systematically enhances both coverage and defect detection efficacy.

A thorough empirical evaluation on seven Defects4J projects, demonstrating substantial improvements over leading search-based and LLM-only baselines in fault detection and branch coverage.

An in-depth cost-benefit analysis of adversarial iterations, yielding practical

guidelines on iteration limits, resource consumption, and termination thresholds for industrial application.

By pioneering the integration of large language models, mutation testing feedback, and adversarial multi-agent dynamics, this work lays the groundwork for next generation automated test-generation systems that combine human-like readability with rigorous coverage and defect detection capabilities.